

Master's Thesis

# Synthetically Extending a Dataset to Improve Machine Learning Prediction

*An Application in Tabular Marketing Data*

Gijs W.M. de Bruin

Student number: 13474936  
Date of final version: August 15, 2023  
Master's programme: Econometrics  
Specialisation: Data Analytics  
Supervisor: Dr. Y. He  
Second reader: Dr. J.D.P. Depersin

UNIVERSITY OF AMSTERDAM  
FACULTY OF ECONOMICS AND BUSINESS  
AMSTERDAM SCHOOL OF ECONOMICS



## Abstract

Good machine learning (ML) algorithms start with good data to train on, but this data is not always available. A potential way to solve this problem is by generating synthetic data to complement a real training dataset. This approach has already been used to boost ML performance in for example the field of computer vision (Frid-Adar et al., 2018; Borkman et al., 2021), but the subject remains largely unexplored when it comes to its application in business and economics. Here, we consider a tabular, binary classification dataset describing customer responses to a marketing campaign. We investigate whether we can use a conditional tabular generative adversarial network (CTGAN) to create synthetic data that improves the performance of a random forest (RF) algorithm trained on a combination of real and synthetic data. Two approaches are considered: (i) an “internal” approach, where the real train set is complemented by data from a CTGAN generator that was trained on the real train set itself, and (ii) an “external” approach, where the largest part of the real train set is considered to be outside data which is only accessible to the CTGAN. In this approach only a small subset of the train set is directly accessible, and this set is combined with synthetic data generated by the CTGAN. We use both methods to train RF algorithms, and employ a bootstrap approach to get an estimate of the RF performance metrics. For approach (i), no significant increase or decrease in RF performance is observed. Approach (ii) does show a considerable increase in RF performance: when adding a synthetic dataset with 99k observations to a small real data set with 165 observations, the absolute increase in accuracy, precision, recall, F1 and AUC-ROC is 1.7%, 4.7%, 18%, 18.7% and 3.5%, respectively. The boosted performance is on par with using the full real training dataset for training the RF, indicating that real data can be replaced by synthetic data without compromising on the RF’s performance. Moreover, it is found that the increase in precision and recall has a strong effect on the profitability of the marketing campaign that we consider, leading to a profit increase of 99% for the particular case when adding synthetic data to the 165 real observations.

# Contents

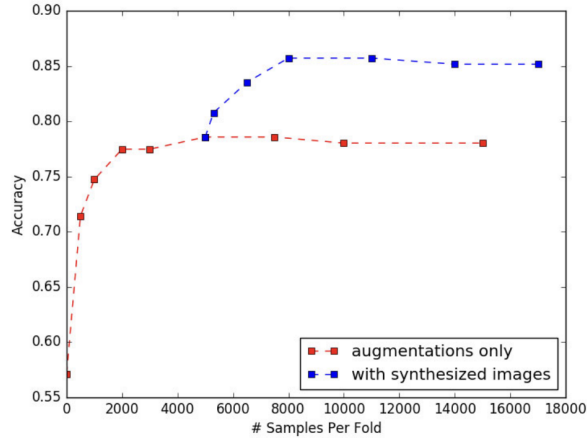
|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>1. Introduction</b>                                               | <b>2</b>  |
| <b>2. Literature Review</b>                                          | <b>4</b>  |
| 2.1 Synthetic Data Generation: from Images to Tables . . . . .       | 4         |
| <b>3. Data</b>                                                       | <b>6</b>  |
| 3.1 Categorical Features . . . . .                                   | 7         |
| 3.2 Numerical Features . . . . .                                     | 8         |
| 3.3 Binary Features . . . . .                                        | 9         |
| <b>4. Methodology</b>                                                | <b>10</b> |
| 4.1 The Conditional Tabular Generative Adversarial Network . . . . . | 10        |
| 4.2 The Random Forest Algorithm . . . . .                            | 13        |
| 4.3 The Data Flow . . . . .                                          | 14        |
| 4.3.1 Internal Approach . . . . .                                    | 14        |
| 4.3.2 External Approach . . . . .                                    | 15        |
| 4.4 Understanding and Mitigating Randomness . . . . .                | 16        |
| 4.5 Assessing RF Performance . . . . .                               | 17        |
| <b>5. Results and Discussion</b>                                     | <b>19</b> |
| 5.1 Visual Analysis . . . . .                                        | 19        |
| 5.2 Results for the Baseline Performance . . . . .                   | 20        |
| 5.3 Results for the Internal Approach . . . . .                      | 22        |
| 5.4 Results for the External Approach, Case I . . . . .              | 24        |
| 5.5 Results for the External Approach, Case II . . . . .             | 26        |
| <b>6. Conclusion</b>                                                 | <b>29</b> |

# 1. Introduction

In 2006, mathematician Clive Humby coined the expression “data is the new oil”. Since then we increasingly saw how striking this analogy is: like oil, data fuels economic growth and it is being sold like any other commodity. Especially in the light of the recent AI boom, it is clear how important data is and how much more important it will become. However, not everyone has access to the data they need. Some datasets may be too small, incomplete, or one may not have the means, capability or permission to harvest and use certain data. Consider for example an AI start-up that wants to train their new AI model: as the company is just starting out, they will have no or little data to train their AI model. Maybe there are other companies that have the data they need, but they won’t share it due to privacy regulations. The start-up may try and go gather data themselves, but this may take a lot of time and could involve bringing in additional employees or external parties at considerable costs. This raises the question whether there is a cost-effective way to solve the problem of data unavailability.

Enter the era of generative models. The past years were marked by great advances in the research on these type of models. Well known generative models are the image generator DALL-E and text generators like ChatGPT. These models are able to synthesize completely new data that does not directly relate to any real-world observations. We call this data **synthetic data**. The ability of generative models to create synthetic data begs the question whether we can use them to create new data in situations where there is a challenge surrounding data availability. It turns out that synthetic data is already being used to this end. An established application is in the case of privacy sensitive data, like medical records. Sensitive medical data can be replaced by synthetic data such that the dataset does not contain personally identifiable information anymore, while still preserving the underlying statistical structure of the original, sensitive dataset (Weldon, Ward, & Brophy, 2021). This allows the synthetic data to be used for machine learning tasks that would otherwise not be allowed using the real dataset. The intuition here is clear, assuming that one is able to create a synthetic dataset that successfully mimics the underlying characteristics of the real dataset. Another established application of synthetic data is found in computer vision: when training a synthetic data generator on an imagery dataset, it is possible to boost ML performance by training the model on a combination of real data and synthetic data from the generator. Figure 1.1 on the next page shows results from Frid-Adar et. al’s (2018) research into synthetically extending a CT-scan imagery set to boost the performance of their classification algorithm, we will further elaborate on their research in the literature review section. The intuition here may be less clear than in the privacy use case: we appear to create a more valuable dataset based on the original dataset alone. The hypothesis is that synthetic data can boost performance as it will add more variability to the feature space (regularization) and potentially fills certain areas in the feature space that are wrongly underrepresented in the real train set. Interestingly, for tabular datasets we do not find examples in literature where synthetic data boosts ML performance. Manousakas and Aydınoğlu (2023) point out that advances in generative models for tabular data lag behind advances in image, audio and text data due to unique challenges surrounding generative modelling for tabular data. We will further elaborate on this in the literature review.

Synthetic data usage may overshadow the use of real data in the near future. Technological research firm Gartner projects that synthetic data will account for over 60% of all AI training data by 2024. Investors are jumping on the bandwagon as well and synthetic data start-ups are raising considerable amounts of funding. Some companies focus on imagery data, like Datagen (\$72M funding) and Synthesis AI (\$21M in funding), others on tabular data, like Gretel.ai (\$68M funding) and Mostly AI (\$31.1M funding). Clearly, synthetic data is on the rise, and taking a quick look at some of the websites of these companies leaves the impression that synthetic data is *the* universal solution to



**Figure 1.1:** Figure from Frid-Adar et al. (2018) showing how the accuracy of a image classification model trained on a real dataset extended by straightforward augmentation (red) can be significantly improved when GAN-generated synthetic data is added (blue). In both cases, the performance increase saturates when more and more augmented/synthetic data is added.

challenges surrounding data availability. However, it is good to distinguish between the use cases and data types considered: the privacy use case is not the same as the data extension use case, and the application to imagery data does not yield the same results as the application to tabular data (Manousakas & Aydınoğlu, 2023). A use case that remains largely unexplored in literature is whether one can **synthetically extend a tabular dataset to boost ML performance**. This will be the central use case we focus on in this thesis. In business and economics we usually deal with tabular datasets. There are certain caveats to consider when generating synthetic tabular data. For example, tabular data usually contains both numerical and categorical features that have to be modelled simultaneously, there may exist imbalance in the categorical features and numerical features may be non-normally distributed.

In this thesis we consider the tabular dataset of the Brazilian food delivery company iFood, available on Kaggle.com. The dataset relates a consumer’s characteristics to whether he or she responded to a marketing campaign, the latter being the binary target label. The cleaned dataset contains 2205 observations for 29 features. Synthetic data is generated using a conditional tabular generative adversarial network (CTGAN), which is a GAN model optimized for handling tabular data. We take two different approaches to assess if we can synthetically boost ML performance: (i) the “internal” approach, where the real train set is complemented by synthetic data from a CTGAN that was trained on the real train set itself, and (ii) an “external” approach, where the largest part of the real train set is considered to be outside data which is only accessible to the CTGAN. Only a small subset of the train set is directly accessible, and this set is combined with synthetic data generated by the CTGAN. For both cases we investigate how adding synthetic data affects the performance of a random forest algorithm trained on a combination of real and synthetic data.

The remainder of this thesis is organized as follows: in chapter 2, we review relevant literature related to synthetically extending datasets with the goal to boost ML performance. In chapter 3, we present the tabular marketing dataset of interest for this thesis. In chapter 4, the methodology is presented. Here, we introduce the theory behind the CTGAN and explain the internal and external approach. The results are presented and discussed in chapter 5 and the conclusion is given in chapter 6.

## 2. Literature Review

Van der Schaar et al. (2023) state that synthetic data can be used to tackle the three most prevalent issues affecting training datasets: data scarcity, privacy, and bias (fairness). Scarcity refers to small datasets or sets that miss certain data (e.g. data from remote areas that can not be reached). Privacy refers to data that can not be used or shared due to privacy considerations. Bias refers to data that inadvertently misrepresents information about e.g. certain groups of people. In this research we will focus on the use case of synthetically extending a small (scarce) dataset. This particular use case remains largely unexplored for tabular data in the literature. It is therefore of interest to also look at alternative data types in our literature review in order to understand how synthetic extension can boost ML performance, and whether we can expect similar results for tabular data.

### 2.1. Synthetic Data Generation: from Images to Tables

One of the promising applications of synthetic data is in the field of computer vision. In the introduction we showed figure 1.1 from research by Frid-Adar et al. (2018), who show that they can improve the classification of liver lesions in CT-imagery data. Their original classification dataset is balanced and contains 182 images which can contain one of three different lesions. They train CNN models for classification using real and synthetic data and use 3-fold CV to assess if adding synthetic data improves machine learning performance. The original, real dataset contains 182 images, and it is extended by straightforward data augmentation (flip, rotate, scale, etc.). Without adding any synthetic data, they observe that straightforward data augmentation reaches an accuracy of 78.6% for 5000 samples per fold or more – adding more augmented samples doesn’t improve accuracy after 5000 total samples. Next, they add synthetic data to the real dataset. The synthetic data was generated using a Generative Adversarial Network (GAN). They find that if they add 3000 synthetic observations to the existing 5000, the accuracy increased and saturates at 85.7%, as we observed in figure 1.1. Saturating improvement when adding more and more synthetic data is something we see in other literature as well (Borkman et al., 2021). Aside from increased accuracy, the authors report that sensitivity and specificity grow from 78.6% and 88.4% to 85.7% and 92.4%, respectively. Improvements like this are very significant considering more than 20% of the population is affected by benign liver lesions.

Another compelling use case of synthetic data can be found in recent work by Gunasekar et al. (2023). They train a Large Language Model (LLM) called phi-1 that generates Python code. The model is trained on “textbook-like” data, part of which is real and part of which is synthetically generated by a GPT-3.5 model. The LLM they create outperforms other state-of-the-art models significantly while using only a fraction of the training data and parameters. For example, StarCoder (2023) is a state-of-the-art model with 15.5B parameters trained on a 1T token dataset. Phi-1 consists of only 1.3B parameters and was trained on a dataset of 7B tokens. The former model scores 33.6% on HumanEval – which is a common benchmark for Python LLM’s – the latter scores 50.6%. The authors argue that, like human learners, the LLM learns better when it is provided high quality data, which is “clear, self-contained, instructive, and balanced.” In their work they intentionally select real data and generate synthetic data that has these high-quality characteristics.

The examples mentioned above concern forms of unstructured data like image and text data. Although we call these data types unstructured because they are not as neatly organized as for example tables, these data types do have a high degree of local structure. That is to say that nearby data points are usually closely related and have similar values. Note for example that an image of a dog still has the same meaning to us when it is rotated, enlarged, or when the dog is moved to another

spot in the picture. For Machine Learning models like the CNN the meaning of the pictures also doesn't change, we refer to this as the model being invariant. Due to this invariance principle related to imagery data there are a lot of possibilities to create new data based on the original dataset, and it intuitively makes sense that synthetically extending imagery datasets can improve ML performance. Note that tabular data also exhibits local structure. Similar tabular observations will be closer to each other, but not in the two dimensional spatial sense like for imagery data, but in the high dimensional feature space. In our understanding, the problem for tabular data is that the local structure in this high dimensional space is very hard to capture. Consider the following intuitive example: moving up a white dog on a picture by 10 cm can simply be done by moving up the highly correlated white pixels representing the dog in the picture. Now consider a tabular observation with 30 features, and we change the gender category in a tabular observation from male to female. How would the rest of the features change? There are many intricate correlations in the high dimensional feature space that are not easily captured. Intuitively, this is also why there is less opportunity to create synthetic data for tabular sets that would meaningfully increase the value of the dataset, any new observations should be very similar to the existing dataset to be sure we don't change the underlying structure of the dataset. Compare this again to the picture, where we can make a significant change by moving the dog from the bottom left to the top right, while still preserving the meaning of the picture. There are some other challenges surrounding the generative modelling of tabular data, these are the most important ones (Manousakas, & Aydınoğlu, 2023; Xu et al., 2019):

- **Complicated local structure:** Tabular data contains intricate correlations in high dimensional feature space, and these relationships are difficult to model.
- **Co-existence of numerical and categorical features:** Tabular data usually has both categorical and numerical features. It is more difficult to learn a joint distribution over these inherently different features. A problem usually not present in e.g. imagery data.
- **Non-Gaussian and Multimodal Distributions:** The pixel values in images follow a Gaussian-like distribution (Xu et al., 2019). Continuous tabular data is usually non-Gaussian which complicates modelling as an alternative distribution needs to be learned while there usually is not enough data available to get a detailed view of the distribution.
- **Imbalanced categorical columns:** Tabular datasets usually have categorical columns that are (highly) imbalanced. Missing a minority class can cause a generator to completely ignore parts of the real feature space. This is a phenomenon we call mode collapse. Note that this problem is not unique to tabular data, e.g. imagery data may also be imbalanced.

Although generative modelling for tabular data is met with challenges, there are still interesting applications to be found in the literature. A notable example comes from Yoon et al. (2018), who show that medical datasets that have only partially overlapping feature spaces can be merged together by filling in the missing values using a modified GAN. This outperforms the benchmark of naively combining the two datasets and deleting the features that don't overlap. Note that this is a rather specific application of synthetic data. Other successful applications include balancing highly imbalanced datasets with synthetic data (Dina et al., 2022). Interestingly, the more straightforward application of synthetically extending a generic tabular dataset remains largely unexplored in literature. This is what we will focus on in this thesis. The hypothesis is that synthetic data can boost performance as it will add more variability to the feature space (regularization) and potentially fills certain areas in the feature space that are wrongly underrepresented in the real train set. Our dataset is presented in the next chapter.

### 3. Data

The data considered in this research is a digital marketing dataset from the Brazilian online food delivery platform iFood, which was found on Kaggle.com. The full original dataset contains 2240 observations for 29 features. The features are described in the table below.

| No. | Variable            | Description                                                        |
|-----|---------------------|--------------------------------------------------------------------|
| 1   | AcceptedCmp1        | 1 if customer accepted the offer in the 1st campaign, 0 otherwise  |
| 2   | AcceptedCmp2        | 1 if customer accepted the offer in the 2nd campaign, 0 otherwise  |
| 3   | AcceptedCmp3        | 1 if customer accepted the offer in the 3rd campaign, 0 otherwise  |
| 4   | AcceptedCmp4        | 1 if customer accepted the offer in the 4th campaign, 0 otherwise  |
| 5   | AcceptedCmp5        | 1 if customer accepted the offer in the 5th campaign, 0 otherwise  |
| 6   | Response (target)   | 1 if customer accepted the offer in the last campaign, 0 otherwise |
| 7   | Complain            | 1 if customer complained in the last 2 years                       |
| 8   | DtCustomer          | Date of customer's enrolment with the company                      |
| 9   | YearBirth           | Customer's birth year                                              |
| 10  | Education           | Customer's level of education                                      |
| 11  | Marital             | Customer's marital status                                          |
| 12  | Kidhome             | Number of small children in customer's household                   |
| 13  | Teenhome            | Number of teenagers in customer's household                        |
| 14  | Income              | Customer's yearly household income                                 |
| 15  | MntFishProducts     | Amount spent on fish products in the last 2 years                  |
| 16  | MntMeatProducts     | Amount spent on meat products in the last 2 years                  |
| 17  | MntFruits           | Amount spent on fruits products in the last 2 years                |
| 18  | MntSweetProducts    | Amount spent on sweet products in the last 2 years                 |
| 19  | MntWines            | Amount spent on wine products in the last 2 years                  |
| 20  | MntGoldProds        | Amount spent on gold products in the last 2 years                  |
| 21  | NumDealsPurchases   | Number of purchases made with discount                             |
| 22  | NumCatalogPurchases | Number of purchases made using catalogue                           |
| 23  | NumStorePurchases   | Number of purchases made directly in stores                        |
| 24  | NumWebPurchases     | Number of purchases made through company's web site                |
| 25  | NumWebVisitsMonth   | Number of visits to company's web site in the last month           |
| 26  | Recency             | Number of days since the last purchase                             |
| 27  | Z_CostContact       | Campaign cost per targeted customer (=3)                           |
| 28  | Z_Revenue           | Campaign revenue per (correctly) targeted customer (=11)           |
| 29  | Customer ID         | Identification Number of Customer                                  |

**Table 3.1:** All features in the original dataset. Z\_CostContact, Z\_Revenue and Customer ID are dropped from the training dataset as they do not add any information for classification. Three new features are engineered: (i) MntAgg, which aggregates the purchased amounts over all product types, (ii) AcceptedCmpAgg, which aggregates the responses for all marketing campaigns and (iii) MntNormalProducts, which aggregates the amount of normal products bought.

We start cleaning the data by dropping the 24 observations that contain missing values, and the three columns that add no information for classification (these are the user ID, the cost per customer and the revenue per customer – the latter two are the same for every observation, \$3 and \$11 respectively). We will consider the cost and revenue variables later when relating RF performance to profit/loss. Next to dropping some instances, we will engineer 3 new features that contain interesting information about consumer characteristics: (i) a feature that combines the number of products bought in different categories (MntAgg), (ii) a feature that aggregates the total number of campaigns accepted (AcceptedCmpAgg) and (iii) a feature for the amount of “normal” products bought (MntNormalProds). Now that we did initial cleaning, we will consider the categorical, numerical and binary features more in-depth.

### 3.1. Categorical Features

We visualize the categorical variables in figure 3.1 below. In the first two figures we see how the observations are distributed over the education and marital status categories. There is some imbalance with the “basic” and “widow” categories being quite a bit smaller than the other categories. The third figure relates the education and marital status categories. In the bottom two figures we see how the responses in the target variable are distributed for the different categories. There is a slight imbalance for some categories.

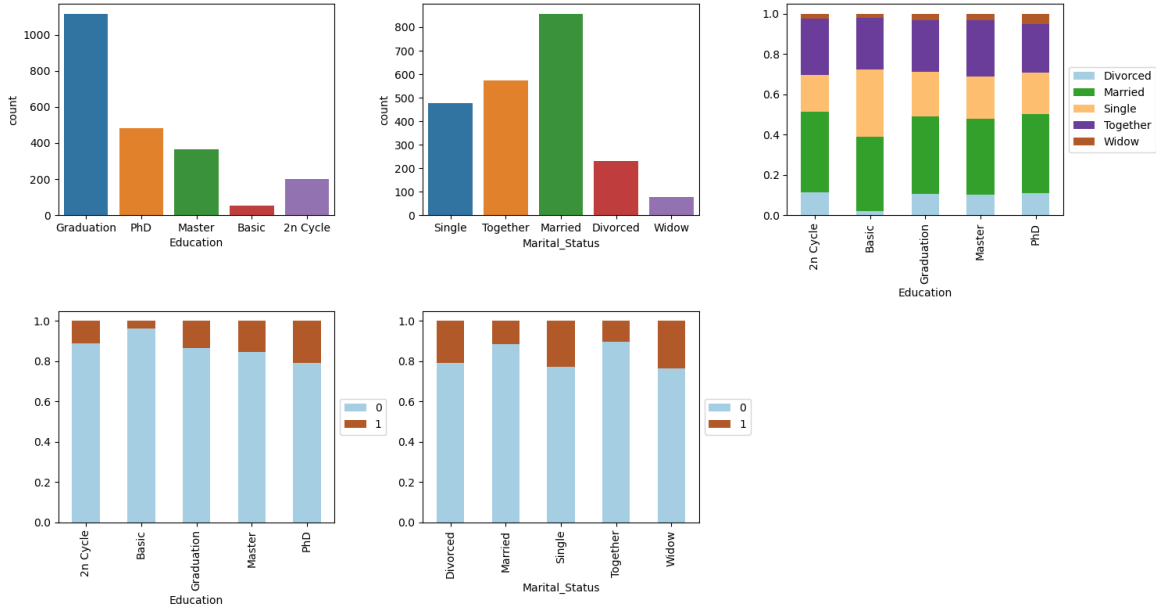
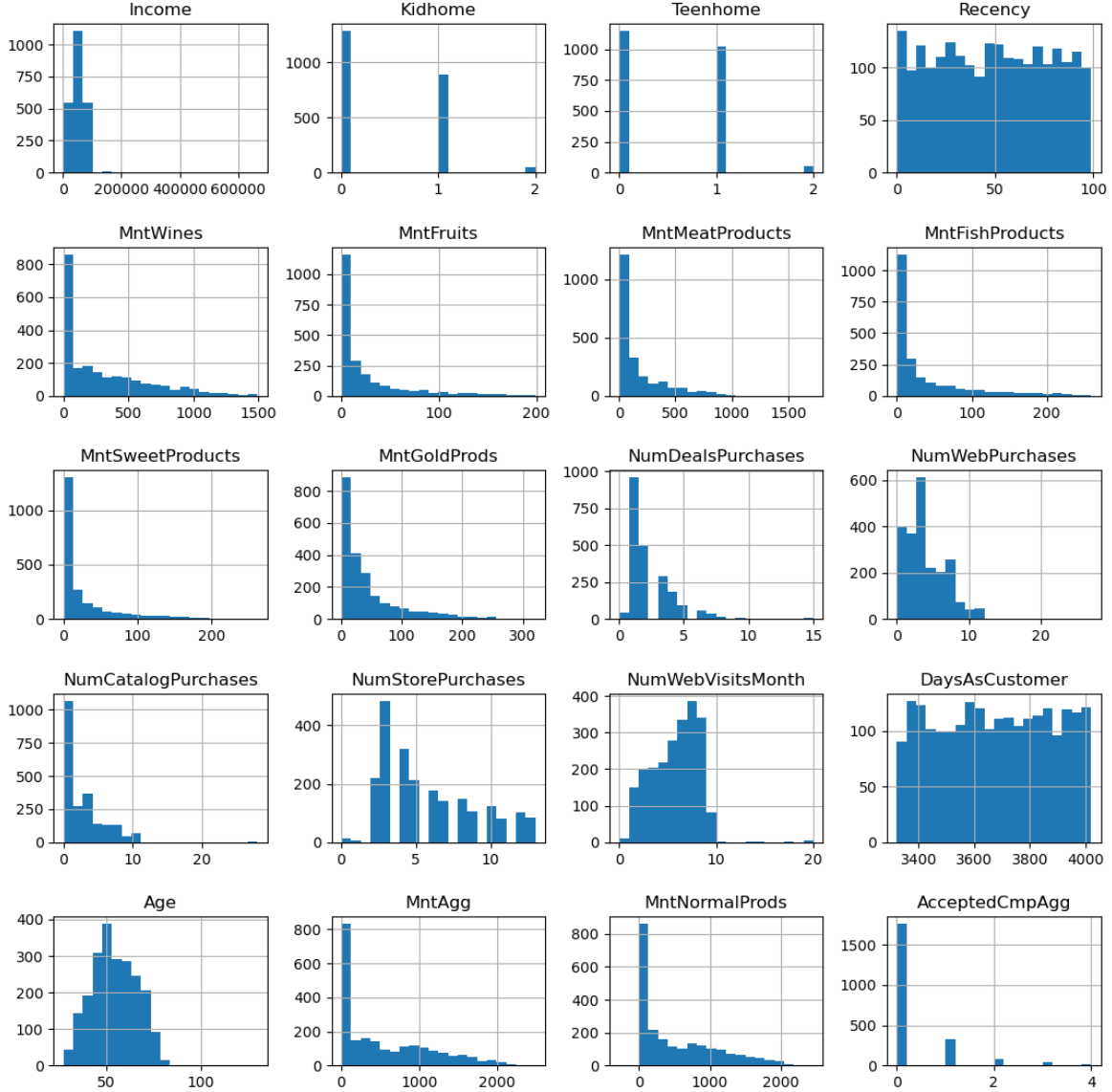


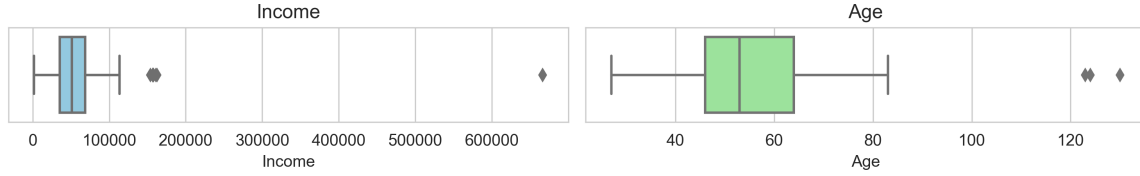
Figure 3.1

## 3.2. Numerical Features

Histograms of the numerical features are provided below. The histograms allow us to see if there are distributional characteristics that need to be taken into account and if there are outliers that need to be removed. From the histograms, we see that there likely are outliers for income and age. We check this by making a boxplot, see figure 3.3. In the boxplot we indeed observe outliers in the income and age features. We remove the outliers through the interquartile (IQR) method. A correlation plot is given in figure 3.4.

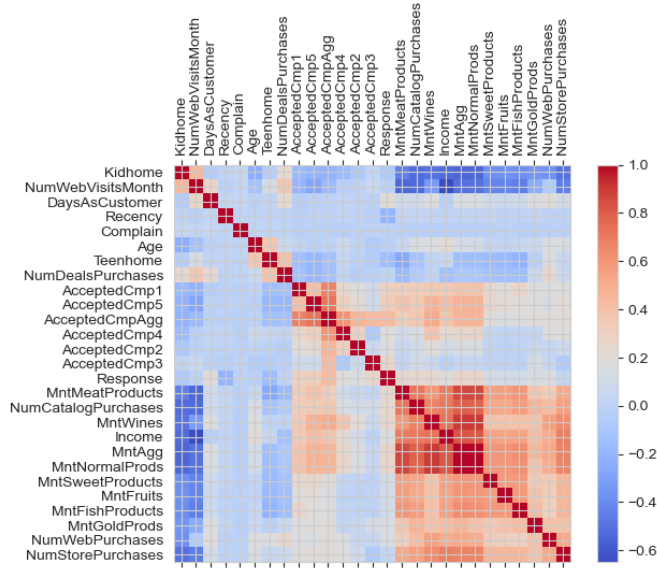


**Figure 3.2:** Histogram of numerical features. Number of bins equals 20. Note how a lot of the distributions are skewed to the right. The skewness suggests that only a small part of our clients buy a lot, most buy a little. We also see that some distributions exhibit multi-modality (e.g. NumWebPurchases, MntAgg, MntNormalProds, Recency and DaysAsCustomer). NumDealsPurchases and NumStorePurchases contain some gaps in the distribution that we are not able to explain.



**Figure 3.3:** Boxplot for the income and age feature. The whiskers indicate the minimum and maximum values of the features (excluding the outliers as defined by the IQR method). The box signifies the 1<sup>st</sup> and 3<sup>d</sup> quartiles, the line inside the box represents the 2<sup>nd</sup> quartile (median). The outliers are the grey points and there are 4 and 3 outliers observed for income and age, respectively.

**Figure 3.4:** Correlation plot for numerical features. Note that MntAgg and MntNormalProds are highly correlated as the second one was based on the first one. Most positive correlation likely comes from income: higher incomes tend to buy more of everything, so product amounts positively correlate with income and between themselves. Same goes for income and the purchase location (Catalog/ Store/Web). Interestingly, the amount of kids in the house correlates negatively with amount of products bought. Note: features for campaign response (among which the target feature, “Response”) do not correlate much with other features.



### 3.3. Binary Features

Now that the dataset is completely cleaned there are 2205 observations left. Let’s take a look at the binary features of the cleaned dataset. As you can see, there is quite some imbalance in most of these features. However, the response (target) variable doesn’t show too much imbalance at 15.1%

|              | Sum Positive Class | % of Total |
|--------------|--------------------|------------|
| AcceptedCmp1 | 142                | 6.44       |
| AcceptedCmp2 | 30                 | 1.36       |
| AcceptedCmp3 | 163                | 7.39       |
| AcceptedCmp4 | 164                | 7.44       |
| AcceptedCmp5 | 161                | 7.30       |
| Response     | 333                | 15.10      |
| Complain     | 20                 | 0.91       |

**Table 3.2:** For each binary feature, it is shown what the total sum of positive (or “1”) responses is, and what this sum is as a fraction of the total 2205 observations in the cleaned dataset

## 4. Methodology

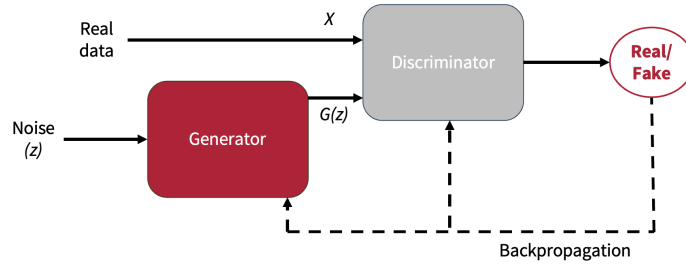
The synthetic data in this thesis will be generated using a GAN-based model: the conditional tabular generative adversarial network (CTGAN), which is especially suited for tabular data. In section 4.1. we will give a high level introduction to GAN's and explain what the additional elements are in the CTGAN approach. In section 4.2 we touch on the choice for random forest as ML algorithm to be trained on the synthetic data. In section 4.3. we present the specifics of how we train the synthetic generator and the RF algorithm. In section 4.4 we explain how we deal with the randomness in our simulation and in section 4.5. we present the performance metrics of interest and relate them to our specific marketing business case.

### 4.1. The Conditional Tabular Generative Adversarial Network

The traditional GAN consists of two parts:

- The generator, which learns to produce synthetic data that looks real.
- The discriminator, which learns to distinguish between the synthetic and real data.

Both models are in a competition that goes back-and-forth: with every run, the generator will create data that increasingly looks like real data, with the goal to trick the discriminator. At the same time, the discriminator improves its ability to distinguish between real and synthetic data with every run. When the GAN is finished training, the discriminator cannot distinguish anymore between the real and synthetic data. The “adversarial” process is described in figure 4.1.



**Figure 4.1:** Schematic of the adversarial game executed by the discriminator and the generator. The models are updated on every iteration through backpropagation.

The initial input of the generator is the random noise vector  $\mathbf{z}$  and it produces an output  $G(\mathbf{z})$ . The discriminator then gets either the generator's  $G(\mathbf{z})$  input or it gets a sample from the real dataset, and tries to predict whether its input was real or synthetic. When the discriminator wrongly predicts whether the input was real or synthetic, it is penalised and its weights and biases are updated through backpropagation. In a similar fashion, the generator is penalised and updated through backpropagation when the discriminator could successfully identify the generated input  $G(\mathbf{z})$  as synthetic. The default model for the generator and discriminator is the multilayer perceptron.

The discriminator performs binary classification, so the go-to loss function is binary cross entropy, which is used in the original GAN paper (Goodfellow et al., 2014). Recall that the binary cross-entropy is given by equation 4.1, where  $y$  is the true class and  $\hat{y}$  is predicted probability.

$$L(y, \hat{y}) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})) \quad (4.1)$$

The discriminator has outputs for real and synthetic,  $D(X) = 1$  and  $D(G(Z)) = 0$  respectively. The two relevant loss equations then are:

$$L(1, D(x)) = -(1 \cdot \log(D(x)) + (1 - 1) \cdot \log(1 - D(x))) = -\log(D(x)) \quad (4.2)$$

$$L(0, D(G(z))) = -(0 \cdot \log(D(x)) + (1 - 0) \cdot \log(1 - D(x))) = -\log(1 - D(G(z))) \quad (4.3)$$

We want to minimize the loss equations, which is the same as maximizing the negative of the loss equations. Combining the equations above we can thus define the following maximization problem:

$$\max_D [\mathbb{E}_{\mathbf{x} \sim p_{\text{real data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (4.4)$$

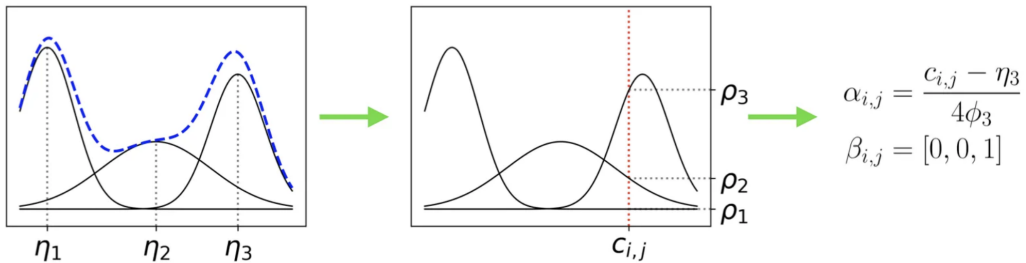
As for the generator, it tries to maximize the probability that the discriminator classifies the synthetic data as being real. It therefore tries to maximize equation 4.3. This can be written as a minimalization problem:

$$\min_G [\mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (4.5)$$

This maximization and minimization problem can be combined to define what Goodfellow et al. (2014) describe as a two-player minimax game with a value function given by:

$$\min_G \max_D [V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{real data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (4.6)$$

In this thesis we will look at tabular data. As mentioned in the literature review, there are some known caveats to tabular data that make generative modelling more challenging. For example, discrete and continuous columns need to be modelled simultaneously, the continuous columns often have a multi-modal and/or non-Gaussian distribution, and the categorical columns can be imbalanced. These are some reasons why straightforward application of the GAN to a tabular dataset falls short to baseline methods, according to Xu et al. (2019). To mitigate the effects of some of these caveats, Xu et al. (2019) propose the ‘‘Conditional Tabular GAN (CTGAN)’’. They use mode-specific normalization in order to overcome non-Gaussian and/or multimodal distributions (relevant for our dataset, as observed in figure 3.2) and they design a conditional generator and training-by-sampling to tackle imbalanced columns (also relevant for our dataset as observed in figure 3.1). Mode specific normalization is explained in figure 4.2 below.



**Figure 4.2:** Figure from Xu et al. (2019) explaining the concept of mode-specific normalization: for each continuous column, a variational Gaussian mixture model is fitted to estimate the number of modes and to find the standard deviations of the respective modes. Three modes are identified in the example above ( $\eta_1$ ,  $\eta_2$  and  $\eta_3$ ). For every value  $c_{i,j}$  in the continuous column  $C_i$ , it is calculated what the probability is for  $c_{i,j}$  to be from the three possible modes. Given the choice from probability densities  $\rho_1$ ,  $\rho_2$  and  $\rho_3$  in the example, we pick the third mode. Then  $c_{i,j}$  is represented as a one-hot vector  $\beta_{i,j} = [0, 0, 1]$  signifying the third mode, and a scalar given by  $\alpha_{i,j} = \frac{c_{i,j} - \eta_3}{4\phi_3}$  to represent the value within the mode.  $\phi_3$  here is the standard deviation of the third mode.

Based on the explanation in figure 4.2, a single observation (row) becomes a concatenation of continuous and discrete columns:

$$\mathbf{r}_j = \alpha_{1,j} \oplus \beta_{1,j} \oplus \dots \oplus \alpha_{N_c,j} \oplus \beta_{N_c,j} \oplus \mathbf{d}_{1,j} \oplus \dots \oplus \mathbf{d}_{N_d,j} \quad (4.7)$$

where  $\oplus$  signifies the concatenation operation and  $\mathbf{d}_{1,j}$  is the one-hot encoded version of a discrete value. This approach better preserves the original distribution after normalization as opposed to e.g. min-max normalization which is commonly used in the standard GAN approach.

Another problem in the traditional application of the GAN is that the generator is fed a vector that is sampled from a standard multivariate normal distribution (MVN). The MVN is eventually mapped to the distribution of the real data through the training iterations of the generator. The problem here is that this approach does not account for possible class imbalance in categorical features. Minority classes are not likely to be randomly sampled, so the generator may not be trained correctly. Just re-sampling the minority class also does not solve this problem, as then the generator learns the re-sampled distribution as opposed to the real data distribution. To tackle this problem, Xu et al. (2019) take a different approach to feeding the generator:

- First, a random categorical column  $D_i$  is chosen. For this column, a probability mass function (PMF) is defined in a way that the PMF of each category is the logarithm of its frequency in the column. A category  $k$  is then picked based on the PMF. Taking the logarithm helps increasing the occurrence of picking minority categories.
- For the category that was picked using the PMF, a conditional vector is defined that represents the category. In order to force the generator to create the category that was picked, a loss function is instated that penalizes the generator if it generates an observation that does not adhere to the conditional vector.

Using this approach, the original distribution can be reconstructed as given in 4.8 below. Here  $D_i$  is the  $i$ th discrete column.  $k$  is a value that this column can take (Xu et al., 2019).

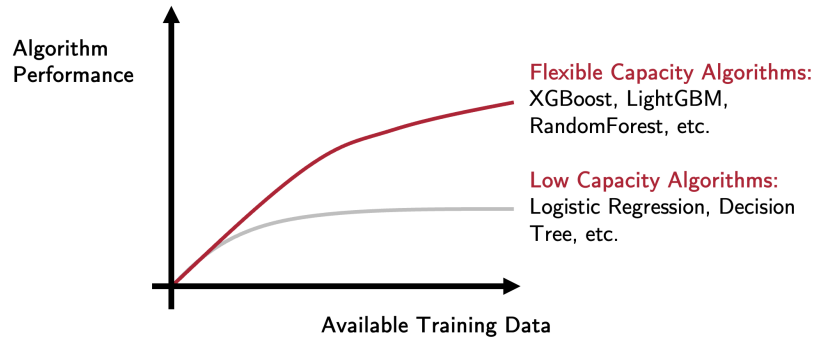
$$\mathbb{P}(\text{row}) = \sum_{k \in D_i} \mathbb{P}_{\mathcal{G}}(\text{row} \mid k) \mathbb{P}(k) \quad (4.8)$$

Lastly, Xu et al. (2019) adapt some elements of the structure of the GAN that was introduced earlier. Most notably, they redefine the discriminator as a “critic” which doesn’t output a binary class, but rather a score for the realness of the generated samples. Due to this change, the loss function used is also different, namely WGAN loss with gradient penalty (Gulrajani et al., 2017). Qian, Cebere and Van der Schaar (2023) created an open-source software package for Python called Synthcity that provides a range of generative models. We will use this package to implement the CTGAN for our dataset.

To assess whether the CTGAN-generated data has a similar structure as the real data, we will present two different dimensionality reduction techniques in the results: principal component analysis (PCA) and t-distributed stochastic neighbor embedding (t-SNE). t-SNE is a non-linear method that preserves only local similarities, therefore it can give other insights than for example PCA, which in principle aims to maintain any distances in the feature space as much as possible. We apply these techniques to both our real and synthetic dataset, and assess whether both approaches yield similar results for the real and synthetic dataset.

## 4.2. The Random Forest Algorithm

Now that we have a generative model that we can use for synthetic tabular data, the next step is to pick a ML algorithm that we will train to see if the synthetic data can boost its performance. As mentioned, if performance improvement occurs for synthetically extending the tabular dataset, it is likely due to the synthetic data introducing additional variety in the feature space and thereby acting as a regularization mechanism. In our case, adding variety means adding more data and thus it is good to consider that not all ML algorithms benefit as much from larger datasets. As indicated in figure 4.3, we can distinguish flexible capacity algorithms and low capacity algorithms. The flexible capacity algorithms have more flexibility that allows the algorithms to capture complicated structures in larger datasets. We therefore opt for a flexible capacity algorithm. A common choice is the random forest algorithm, which is also the algorithm of interest for this thesis.



**Figure 4.3:** Sketch of how algorithm performance relates to the amount of training data for flexible capacity algorithms vs. low capacity algorithms.

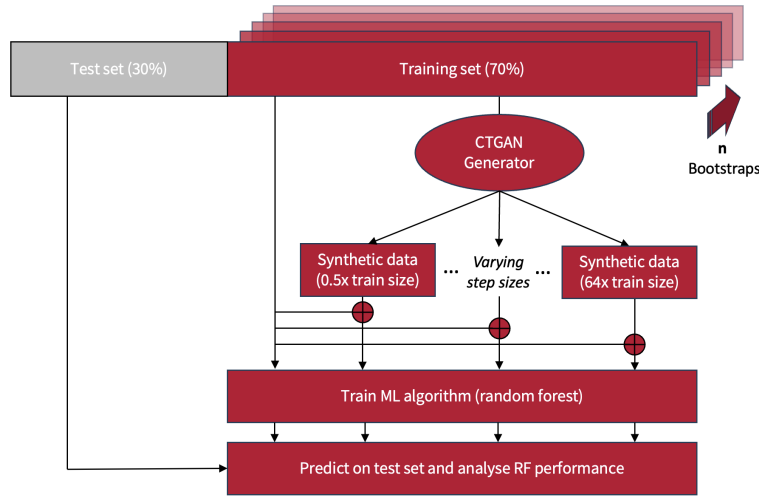
We use a bootstrap approach with which we were able to get robust estimates of RF performance using the bootstrap distributions of the performance metrics. One of the implications of bootstrapping is that it adds a lot of computational intensity to our Python program. The random forest also has hyperparameters that have to be tuned. The usual approach is to do this through e.g. cross validation, but this would further add to the computational intensity of our program if we were to do this for every bootstrap iteration. Instead, we run a single bootstrap iteration with 3 fold CV, and based on this we choose fixed hyperparameters that are suitable for all runs in our program. The maximum depth chosen is `None`, maximum features is `auto`, the criterion used is `gini`, the minimal sample split is `2` and the number of estimators is `500`. We keep the latter parameter rather high as it reduces the variance in the results: the predictions of an RF are averaged over the amount of estimators (trees), so using more estimators reduces the variance.

### 4.3. The Data Flow

Our cleaned dataset undergoes a 70/30 split in a train set and a **hold-out** test set. We will assess the boosting abilities of synthetically extending the train set by taking two different approaches. The first approach is one we will refer to as the “internal approach”. Here, we combine the full trainset with synthetic data from a CTGAN that was trained on the full train set itself. This is in contrast with the “external approach”, where we assume we have direct access to only a small part of the train set, while the CTGAN has access to the full train set and is therefore trained on “external” data as well. We consider the second approach to simulate the situation where a company has a small training dataset and there is an option to buy external data in synthetic form to complement its small dataset. Both approaches are introduced in the following subsections. The subsections mention a bootstrapping approach, which we will elaborate upon in section 4.4.

#### 4.3.1. Internal Approach

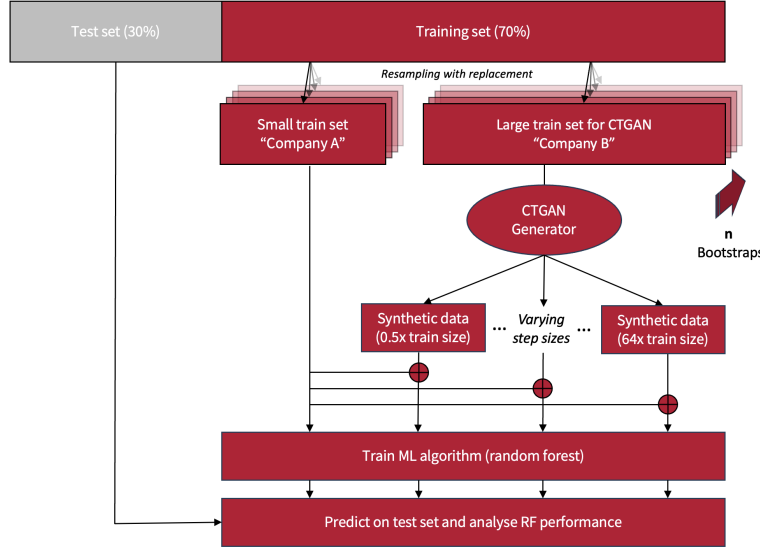
The internal approach is similar to the approach we saw in the computer vision examples discussed (Frid-Adar et al., 2018; Borkman et al., 2021), where a real imagery dataset is extended by synthetic data from a generator that was trained on the real dataset *itself*. Image classification algorithms trained on these extended sets showed boosted performance. The hypothesis is that synthetic data can boost performance as it will add more variability to the feature space (regularization) and potentially fills certain areas in the feature space that are falsely underrepresented in the real train set. Although there is merit to this hypothesis, we don’t expect significant improvement in the internal approach. As discussed in the literature section, tabular data is inherently different from imagery data, and less likely to benefit from synthetic extension.



**Figure 4.4:** Flow of test and training data for the internal approach. The cleaned dataset of size 2205 undergoes a 70/30 split to create a training set and a hold-out test set. Next, we create  $n$  bootstrap training sets through re-sampling the original training set with replacement. For each training set, a CTGAN generator is trained and each generator creates a range of synthetic datasets, varying in size between 0.5x and 64x the original training set size. These synthetic sets are then combined with the real training set as indicated by the  $\oplus$ -symbol, and used to train a range of RF algorithms. The RF performance metrics are obtained by testing on the hold-out test set. Throughout the process above, we use stratified train-test splits, stratified bootstrap resampling, and we also stratify the output of the CTGAN generator by conditioning it to generate synthetic datasets that have the same percentage of 0’s (85%) and 1’s (15%) in the target variable as the original set.

### 4.3.2. External Approach

As we do not expect significant enhancement of the dataset in the internal approach, we will also look at the more compelling “external approach”, described in figure 4.5 below. This approach is best introduced by means of a hypothetical business situation. Imagine a Company A, which has a small tabular marketing dataset for training a classification algorithm, and access to a hold-out test set. Then there is Company B, which has a lot of data on a great variety of subjects, among which data that is similar to the small training set of Company A. Company B sells their data in synthetic form at varying dataset sizes. We would like to see whether Company A would benefit from buying Company B’s synthetic data. This is a hypothetical situation, because in a real-world scenario it is not likely that Company B has a dataset that fits the features and characteristics of Company A’s dataset perfectly. Still, it is an interesting thought experiment and a good way to benchmark the performance of a synthetic data sharing situation like the one described. Also, process flow B can be used to assess the performance of the CTGAN: for a good synthetic data generator, we would see comparable performance between algorithms trained on the full real training set and algorithms trained on a small part of the real train set complemented by a synthetic set that was trained on the full real training set. Put differently, it tells us whether we can *replace* part of the real data by synthetic data without compromising on the RF performance.



**Figure 4.5:** Flow of test and training data for the external approach. The cleaned dataset of size 2205 undergoes a 70/30 split to create a training set and a hold-out test set. Next, we sub-sample the train set with replacement to create a small dataset, and we sample the whole train set with replacement to create a large dataset. For both subsets we create  $n$  different bootstrap subsets. The large train set is used for training the CTGAN generator only. For each bootstrap iteration, a CTGAN generator is trained and each generator creates a range of synthetic datasets, varying in size between 0.5x and 64x the original training set size. These synthetic sets are then combined with the smaller, real training set as indicated by the  $\oplus$ -symbol, and used to train a range of RF algorithms. The RF performance metrics are obtained by testing on the hold-out test set. Throughout the process above, we use stratified train-test splits, stratified bootstrap resampling, and we also stratify the output of the CTGAN generator by conditioning it to generate synthetic datasets that have the same percentage of 0’s (85%) and 1’s (15%) in the target variable as the original set. In our simulations, we consider two sizes for the small bootstrap train set, equal to 15% and 7.5% of the size of the full dataset, respectively. The large bootstrap trainsets for the CTGAN have the same size as the original train set: 70% of the full dataset.

## 4.4. Understanding and Mitigating Randomness

There are several random elements in our methodology that may introduce variance and/or bias in the results. It is important to understand where variance and bias are introduced, and to see whether they can be contained or mitigated. By tackling both, we prevent the estimates from being obscured by random effects or systematic errors, which allows us draw more valuable conclusions about how synthetic data impacts RF performance. Below, we discuss the main sources of variance and bias and describe if and/or how they are contained.

- The initial train-test split is a randomized split with a fixed seed, so every simulation we run considers the same train and test set. Note that for a different train-test split, the RF’s performance estimates would have been different. This is a source of variance in our estimates that we would like to contain. Moreover, if the train-test split is not representative of the true population, it could introduce a bias in the performance metrics. To contain the variance and mitigate the bias, we implement stratification throughout our data flow: the initial train-test split is stratified, the bootstrap resampling is stratified, and the output of the CTGAN generator is stratified by conditioning it to generate synthetic datasets that have the same percentage of 0’s (85%) and 1’s (15%) in the target variable as the original set. Next to this stratification, note that we chose the test to be on the larger side: 30% of the 2205 observations. This also reduces the variance of our estimates compared to using a smaller test set.
- The random forest algorithm also contains inherent randomness. Recall that in the RF algorithm, the predictions are averaged over all estimators/trees, so a high number of estimators will reduce the variability in the prediction. We use 500 estimators, which reduces the variance in the performance estimates considerably.
- The synthetic data generator introduces randomness due to the randomized initialization weights and randomized inputs for the generator and discriminator. Ideally, we would mitigate this by training multiple synthetic data generators for every training dataset and average the predictions over the generators. Van Breugel et al. (2023) refer to this as the “generative ensemble approach”, and state that it tends to improve generative performance due to it being less prone to erroneous generative models (e.g. the generator overfits or underfits – resulting in increased variance or bias, respectively). The ensemble approach will unfortunately not be implemented as it is too computationally intensive when combined with bootstrapping.

Now that we have identified sources of variance/bias and contained/mitigated them where possible, we want to be able to estimate the performance metrics and describe their variance. We will do this by bootstrapping the training set. This allows us to create distributions for the performance estimates, and in turn these distributions allow us to form robust estimates for the performance metrics and to investigate the variability of the estimates. The bootstrap in the internal approach is straightforward: the train set is resampled with replacement to get a bootstrap set that has the same size as the train set. In the external approach, the large bootstrap set for training the CTGAN is the same as the bootstrap train set in the internal approach. To create the small train set, we subsample with replacement from the original training set to create a smaller set. Note that because both the small and large set are created through resampling the original train set, there are datapoints that can end up in both the small and the large training set, i.e. there will be “leakage” of data between Company A and B. Contrary to train-test leakage – which is **not** present in our approach – the leakage between company A and B does not introduce problems. In a real-world situation, Company A will be open to share some datapoints with Company B, because after all any datapoint will help the CTGAN to be trained better. Note that our approach is preferred over choosing fixed train sets for Company A and B, especially for Company A, since this set is small, a fixed train set could introduce skewed results. We therefore want to look at a wide range of train sets for Company A. Due to the computational intensity of training a new CTGAN for every bootstrap, we can only do 200 bootstrap iterations.

## 4.5. Assessing RF Performance

In the results we will consider five performance metrics: the accuracy, the precision, the recall, the F1 score and the AUC-ROC. Since we are considering a business problem (“Which consumers should we target with our marketing campaign?”) we are most interested in the metrics that have a practical interpretation for profit optimization. We will shortly discuss the interpretation of the metrics.

- **Accuracy** indicates the proportion of correct predictions as a percentage of the total predictions. This is a basic insight that we would like to have, but beware that it can be a misleading metric: in our dataset we could classify every customer as a negative responder and still get a considerable accuracy of around 85%. Note also that we can not directly relate accuracy to revenue and costs.
- **AUC-ROC** is the area under the ROC curve. The ROC curve plots the true positive rate against the false positive rate for different prediction thresholds. Therefore, the area under the ROC is a measure for how well the RF algorithm can distinguish between the two classes. An AUC-ROC of 1 signifies it can perfectly discriminate between the two classes, an AUC-ROC of 0.5 signifies that the discrimination ability is no better than random guessing. This is an important metric, but note that it cannot be directly related to revenue and costs.
- **Precision**, calculated as  $\text{True Positive} / (\text{True Positive} + \text{False Positive})$ , gives the probability that we correctly classified a customer given that we classified the customer as a positive responder. If this probability is very low, it means we often predict that customers will respond to the campaign while this is not the case. In marketing campaigns we incur costs for every customer targeted, so if we target a lot of customers that do not lead to sales, we are losing money.
- **Recall**, calculated as  $\text{True Positive} / (\text{True Positive} + \text{False Negative})$ , tells us how many customers we correctly identify as positive responders as a percentage of all customers that responded positively. You want this number to be high, because the more positively responding customers you can identify, the more sales you generate.
- **The F1 score** is the harmonic mean of recall and precision. If either precision or recall is low, the F1 score is also low, so it indicates how the two are balanced. However, it is not directly related to revenue and costs.

The metrics that have a practical interpretation for revenue and costs are precision and recall. The dataset contains information on revenue and costs: the revenue per customer is \$11, and the cost per customer is \$3. From this we can already draw an important conclusion: if we want to make a profit, we need to reach at least one paying customer for every 3.67 ( $= \$11 / \$3$ ) customers that we target. Or put differently, our targeting must be right at least 27.27% ( $= \$3 / \$11 \cdot 100\%$ ) of the time. The latter percentage is the baseline for the precision metric of our algorithm in order to not make a net loss on the campaign. Let’s now try to define revenue and costs as a function of precision and recall for our specific dataset:

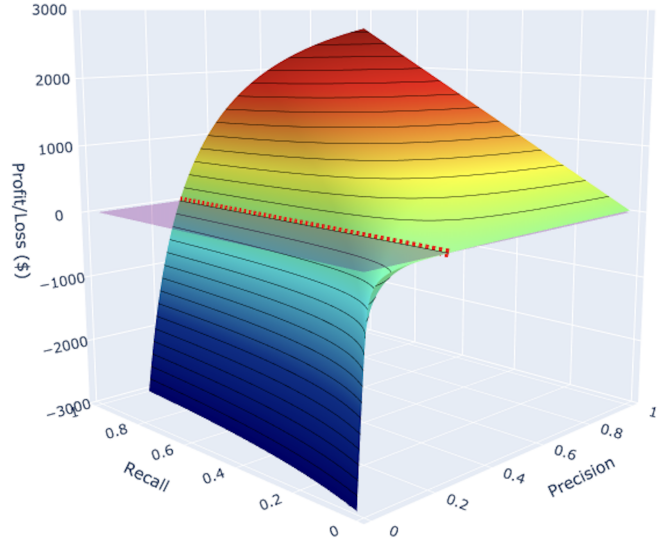
$$\begin{aligned} \text{Revenue} = & 2205 \text{ Customers} \times 15.1\% \text{ Positive response rate} \\ & \times \text{Recall} \times \$11 \text{ Revenue per positive customer} \end{aligned} \quad (4.9)$$

$$\text{Costs} = \frac{2205 \text{ Customers} \times 15.1\% \text{ Positive response rate} \times \text{Recall}}{\text{Precision}} \times \$3 \text{ Cost per target} \quad (4.10)$$

Note how in equation 4.10, the nominator signifies all correctly targeted customers, and by dividing it by the precision we will get the total of all targeted customers. Note that the equations are subject to a boundary condition: when recall is 0, precision is also 0, but costs can still be incurred. The maximum loss is equal to  $2205 \cdot (1 - 15.1\%) \cdot \$3 = \$5616$ . The equations are only valid when at least one target is correctly identified, this implies minimum boundary values for precision and recall which are a little over 0.3% and 0.05%, respectively.

Using the equations above we can calculate:  $\text{Profit (Loss)} = \text{Revenue} - \text{Costs}$ , and plot it as a function of recall and precision, see figure 4.6. The maximum profit is given by the peak in the figure, equal to \$2664. Note that the shaded flat plane – which signifies \$0 profit – intersects the profit surface when precision equals 27.27% as we concluded earlier. An RF algorithm with a precision lower than this is useless. Another important conclusion to be made from the figure is that the derivative (or marginal profit) diminishes as precision increases, while it is constant for increasing recall.

**Figure 4.6:** Surface plot of the potential profit (loss) of the marketing campaign, as a function of recall and precision. The opaque plane signifies the zero-profit level, which intersects the surface at precision equals 27.27%. Note that the potential losses from a bad marketing model (capped at  $-\$5616$ ) far outweigh the potential profits (\$2664) in the specific campaign that we are considering. It is therefore important to get a good image of how the performance metrics affect profit.



In the results section we will plot the performance metrics as a function of the amount of synthetic data added. As mentioned, we will do only 200 bootstraps. This is a relatively small amount, but from the results section we conclude that it is an adequate amount for the purposes of this thesis. To get a robust estimate of the RF performance metric, we report the median of the bootstrap iterations. Taking the median is more robust to outliers in our small bootstrap than e.g. the mean. The corresponding errorbars will be provided in the form of the 2.5% lower and 97.5% upper quantiles. Note that errorbars constructed this way should not be interpreted as a reliable 95% confidence interval (CI). Rather, the errorbars allow us to visually interpret the overlap between the estimates, and tell whether we could reasonably expect an observed improvement in the median performance to be significant. The histograms of the bootstrapped metrics will also be provided. They will be fitted with a normal density and the corresponding mean and standard deviation will be indicated. The histograms give us another visual tool to judge whether performance improvements are statistically significant.

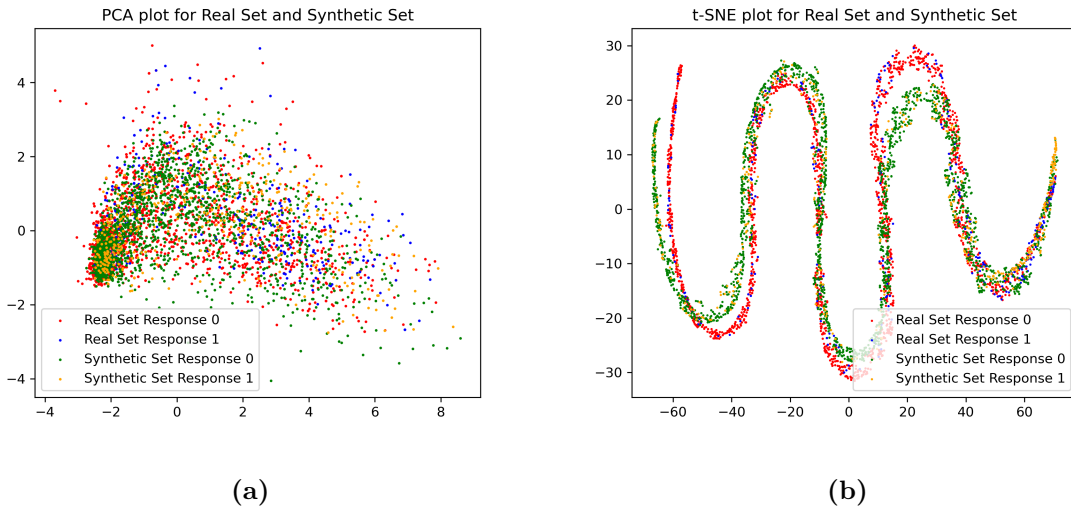
The algorithm performance metrics of the internal and external approach will give a lot of information on whether synthetic data can boost machine learning performance. To further enrich the discussion, we will also set a baseline for performance. We construct a baseline by simulating how the RF performance changes as a function of the amount of real data added – so not including any synthetic data. We do this by sampling a range of real train sets of varying sizes, and plotting the performance of the RF algorithm for the varying train set sizes.

## 5. Results and Discussion

In this chapter, we start by assessing the quality of our synthetic data generator by doing a visual sanity check in section 5.1. Next, we set a baseline for performance in section 5.2, by showing how RF performance increases when extending the training set using only real data. In the remaining sections we will present the results for the internal and external approach.

### 5.1. Visual Analysis

A quick way to check the quality of the synthetic data that we generate is by checking whether the synthetic data looks like the real data set when we describe the data visually. Since the dataset has a feature space with 37 dimensions after dummification we resort to dimensionality reduction techniques: Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE). The associated plots are given in figure 5.1 (a) and 5.1 (b) respectively.

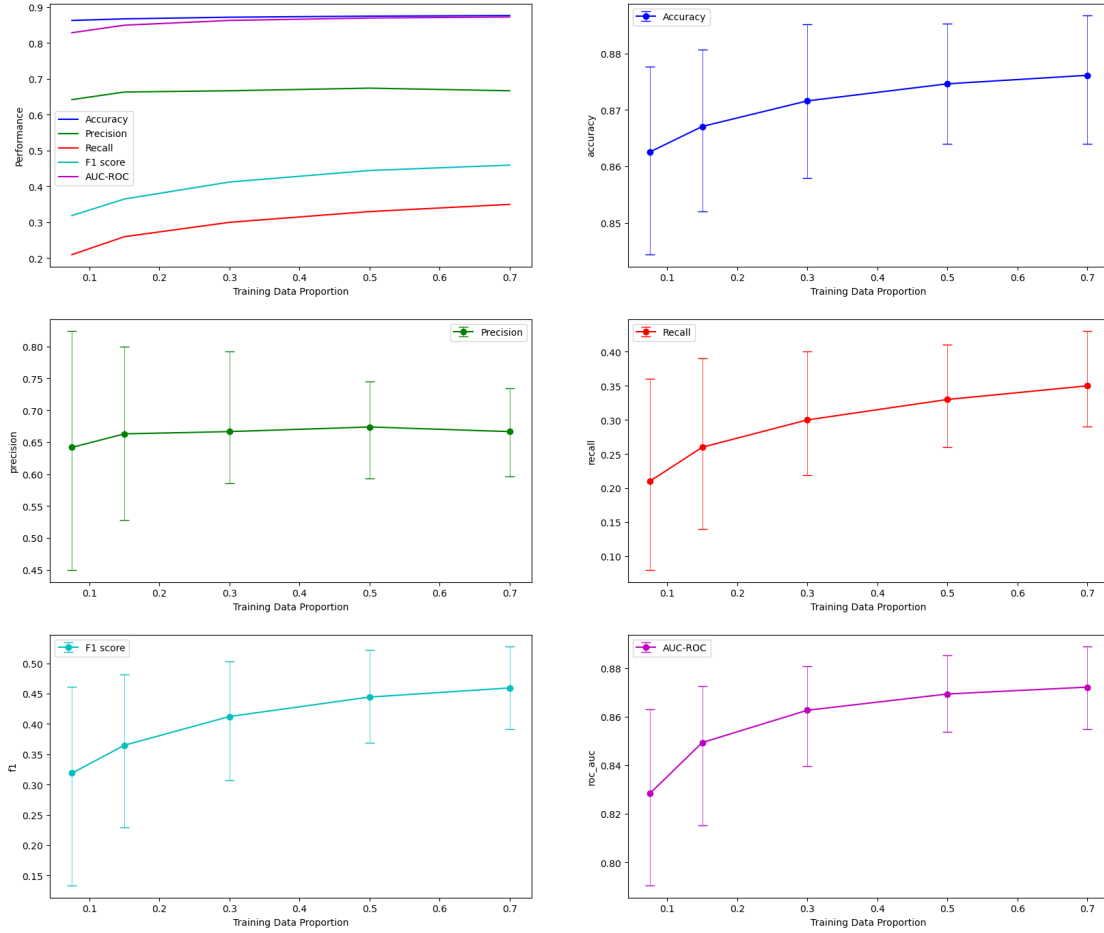


**Figure 5.1:** In the plots above, 2205 real datapoints are plotted using blue dots (for target = 1) and red dots (for target = 0). Similarly, 2205 synthetic datapoints are plotted using orange dots (for target = 1) and green dots (for target = 0). In (a) the PCA plot is given. In (b) the t-SNE plot is given.

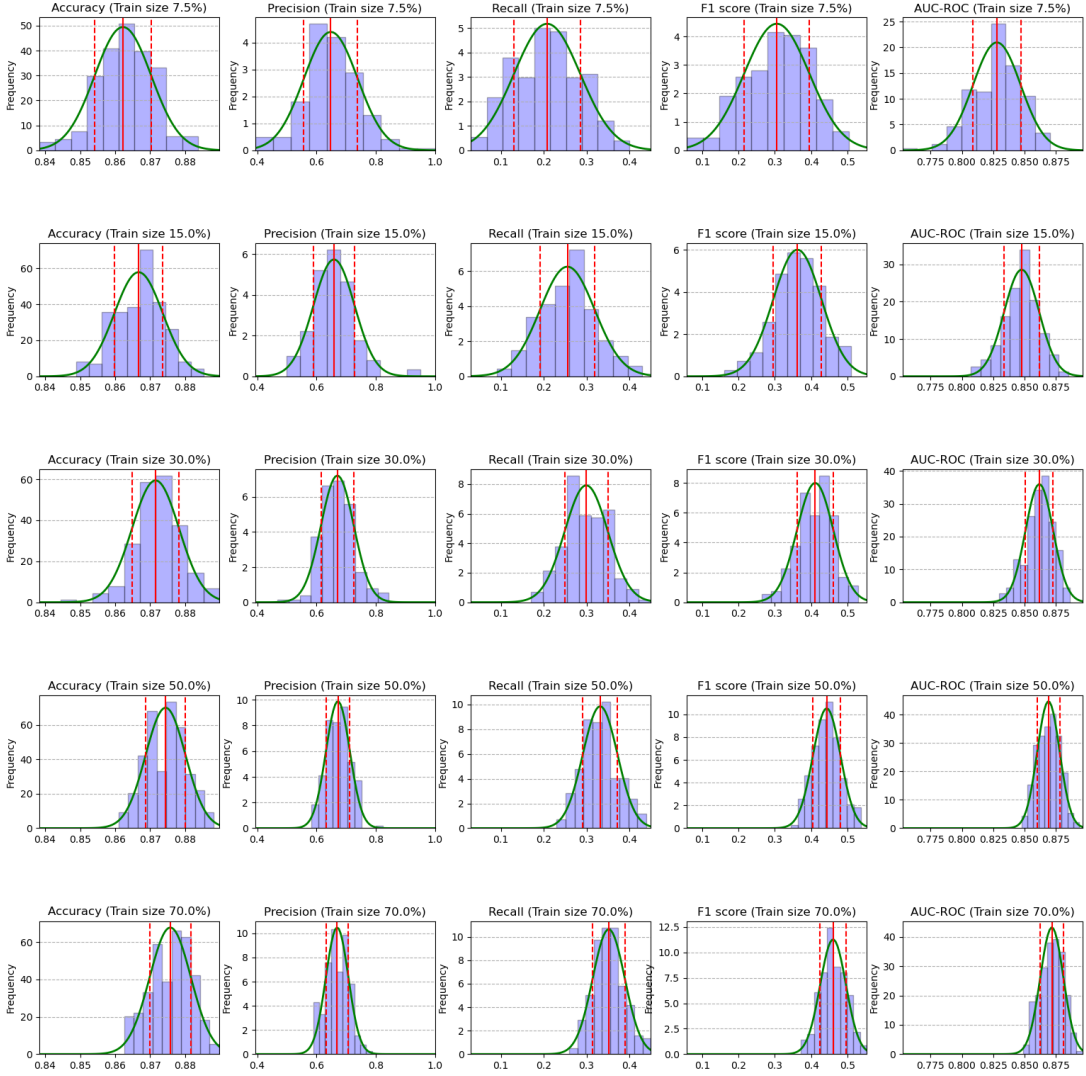
From figure 5.1 it is clear that the synthetic data appears to have the same structure as the real data based on the two visual approaches. For PCA, there is a good overlap of the synthetic and real point clouds at the most dense point on the left. Outside this dense point there is also good overlap, but we do seem to observe more outliers of the real set at the top of the figure, while the synthetic outliers are mostly found at the bottom of the figure. The t-SNE plot shows a similar structure for both the real and synthetic dataset, as expected. Recall that the t-SNE plot only preserves local structure, we therefore are not looking for two perfectly overlapping point clouds, but rather for ones that have roughly the same form. Also note that these plots consider just a single synthetic example. Other examples provide different figures.

## 5.2. Results for the Baseline Performance

Before presenting the results of adding synthetic data, it is good to consider how the RF performance is impacted by the training set size when considering **real data only**. We do this by plotting the RF performance metrics as a function of the amount of training data used, see figure 5.2. For accuracy, precision, recall, F1 score and AUC-ROC, the performance increases when using a larger real training set, and maximal performance is found by using the full training dataset (i.e. 70% of the original dataset). Precision shows a slight improvement in the median, but judging from the errorbars this improvement is very unlikely to be significant. Remember, the errorbars are not a 95% confidence interval, so we have to interpret them cautiously. These observations can also be made when looking at the bootstrap histograms in figure 5.3 on the next page. In table 5.1 we show the absolute performance difference between the smallest (7.5%) and largest (70%) real train set considered.



**Figure 5.2:** Performance metrics for an RF algorithm trained on **real data only**, as a function of the size of the training set. The point values represent using 7.5%, 15%, 30%, 50% and 70% of the total dataset (2205 obs.) for training, respectively. These training subsets were created by sampling with replacement from the original full training set, and this is repeated for 200 bootstrap iterations such that we can estimate performance by taking the median and plotting the 2.5% and 97.5% quantile errorbars. Performance improvement is especially large for recall. Precision does not show a significant improvement.



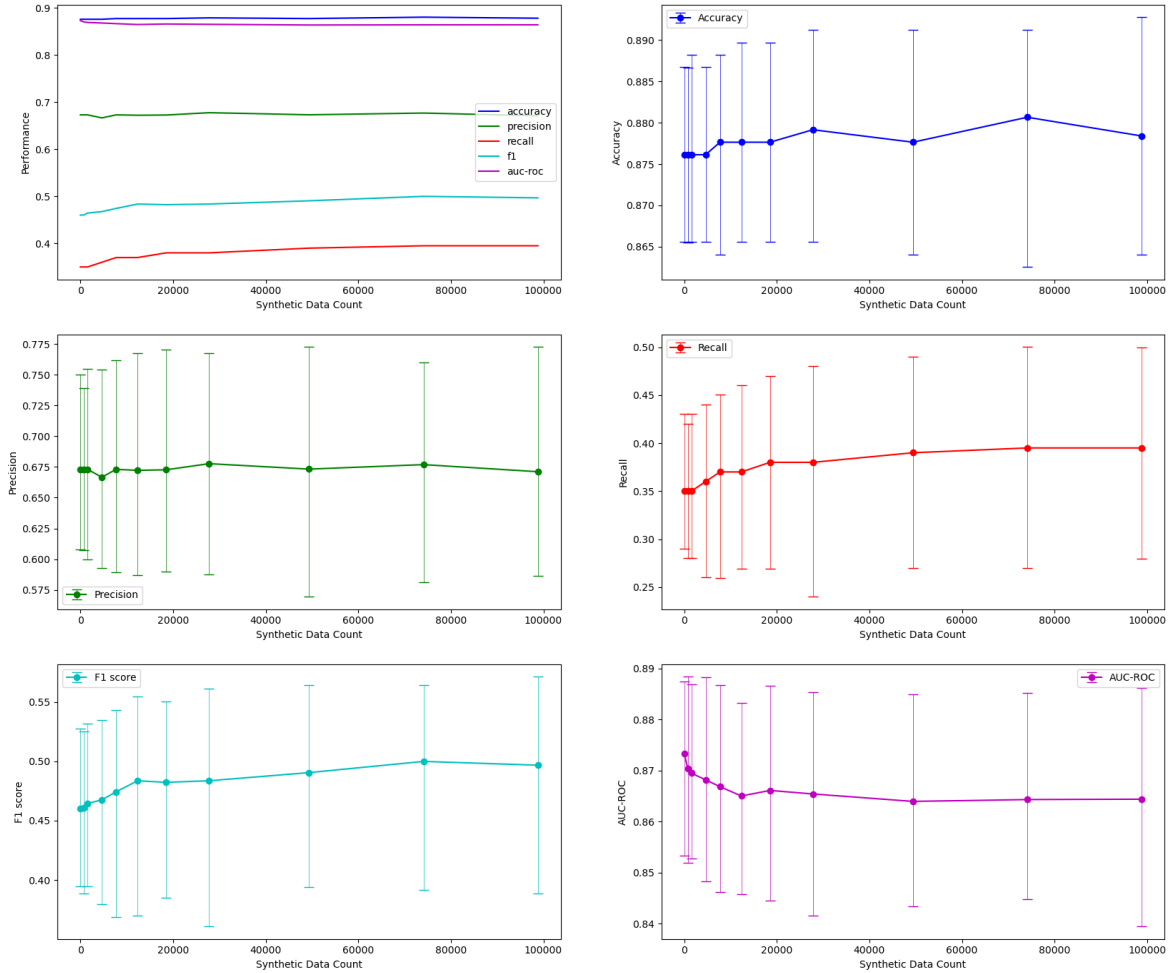
**Figure 5.3:** Histograms of the 200-iteration bootstraps for estimating the RF performance metrics as a function of the amount of real training data used. The x-axes are fixed for every metric, such that we can better observe distributional shifts. A normal density function is fitted in green and the mean and standard deviations are given by the solid and dotted red lines, respectively. We can clearly observe that accuracy, recall, F1 score and AUC-ROC shift to the right when using a larger training set. For precision we only observe the distribution getting narrower, so using more train data will lead to a more consistent performance.

**Table 5.1:** Difference in the median performance between smallest and largest real train set we considered. The improvement in precision is not significant.

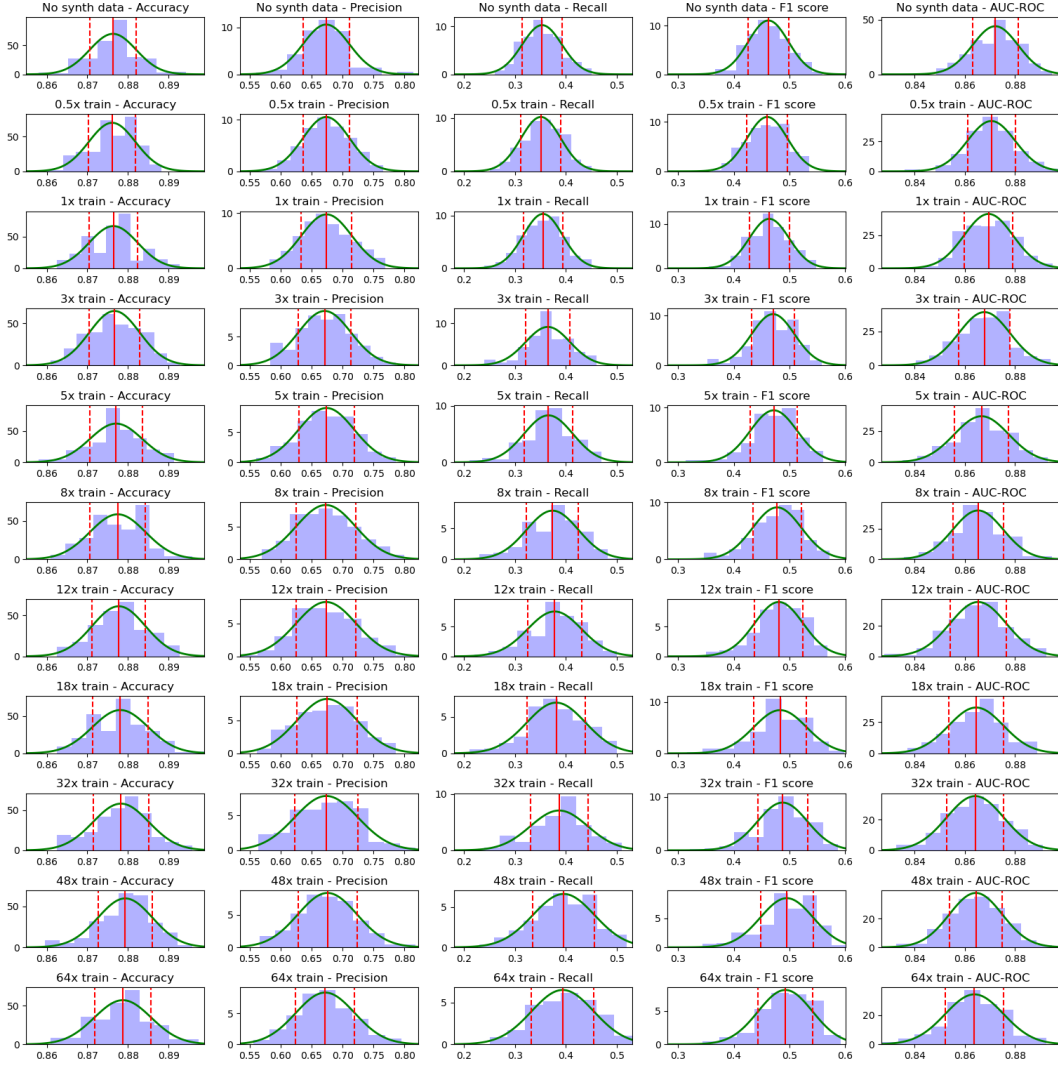
| Train set      | Accuracy | Precision | Recall | F1    | AUC-ROC |
|----------------|----------|-----------|--------|-------|---------|
| 165 real obs.  | 86.3%    | 64.2%     | 21.0%  | 31.9% | 82.8%   |
| 1544 real obs. | 87.6%    | 66.7%     | 35.0%  | 45.9% | 87.2%   |
| Difference     | 1.3%     | 2.5%      | 14.0%  | 14.0% | 4.4%    |

### 5.3. Results for the Internal Approach

The results for the internal approach are given in figure 5.4 below. The corresponding histograms of the bootstrapped performance metrics are given in figure 5.5. Recall that the left-most point corresponds to the situation where only the full real train set is used. The point estimates represent the median value of the bootstraps. Looking at these estimates and their errorbars, we can conclude that there is very likely to be no significant improvement for any of the performance metrics, although recall trends upwards and and AUC-ROC trends downwards. Recall from literature that the boosting abilities of synthetic data – if any – saturate when adding more and more synthetic data. It thus makes sense to look at the situation where no synthetic data is added vs. the situation where the maximum amount of synthetic data is added (64 times the train set, or around 99k observations in our case). We report these situations and their absolute difference in table 5.2.



**Figure 5.4:** Results for the internal approach. The datapoints represent the median over 200 bootstrap iterations, and they are given as a function of the amount of synthetic data added (the synthetic data points represent 0.5x, 1x, 3x, 5x, 8x, 12x, 18x, 32x, 48x and 64x the train set size). The errorbars are provided in the form of the 2.5% lower and 97.5% upper quantiles. The leftmost estimate when synthetic data count equals 0 represents the algorithm that was trained on the real training set only consisting of 1544 ( $= 70\% \cdot 2205$ ) real observations.



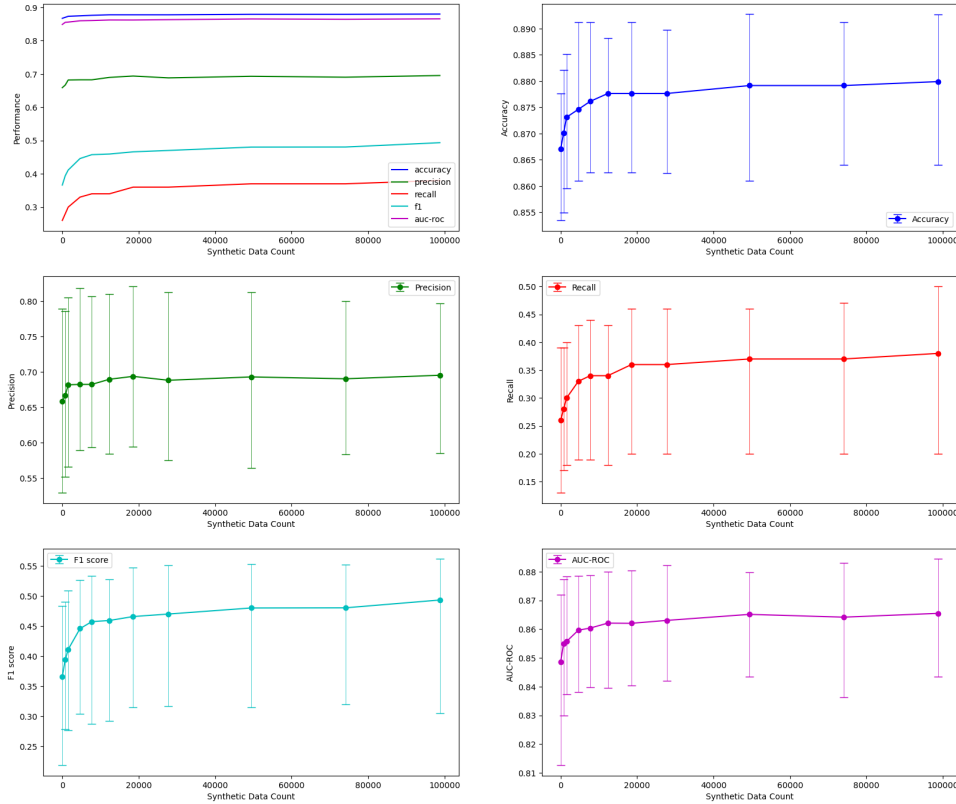
**Figure 5.5:** Histograms of the 200-iteration bootstraps for estimating the RF performance metrics as a function of the amount of synthetic data added in the internal approach. The x-axes are fixed for every metric, such that we can better observe distributional shifts. A normal density function is fitted in green and the mean and standard deviations are given by the solid and dotted red lines, respectively. As we already saw in the errorbars of the line plot, there is a lot of overlap between the distributions, so it is hard to observe any improvements in the internal approach.

**Table 5.2:** Difference in the median performance between using only the real train set with 1544 real observations, versus adding 99k synthetic datapoints to the real train set. **Note:** the difference reported below is not significant based on the errorbars and distributions we observed.

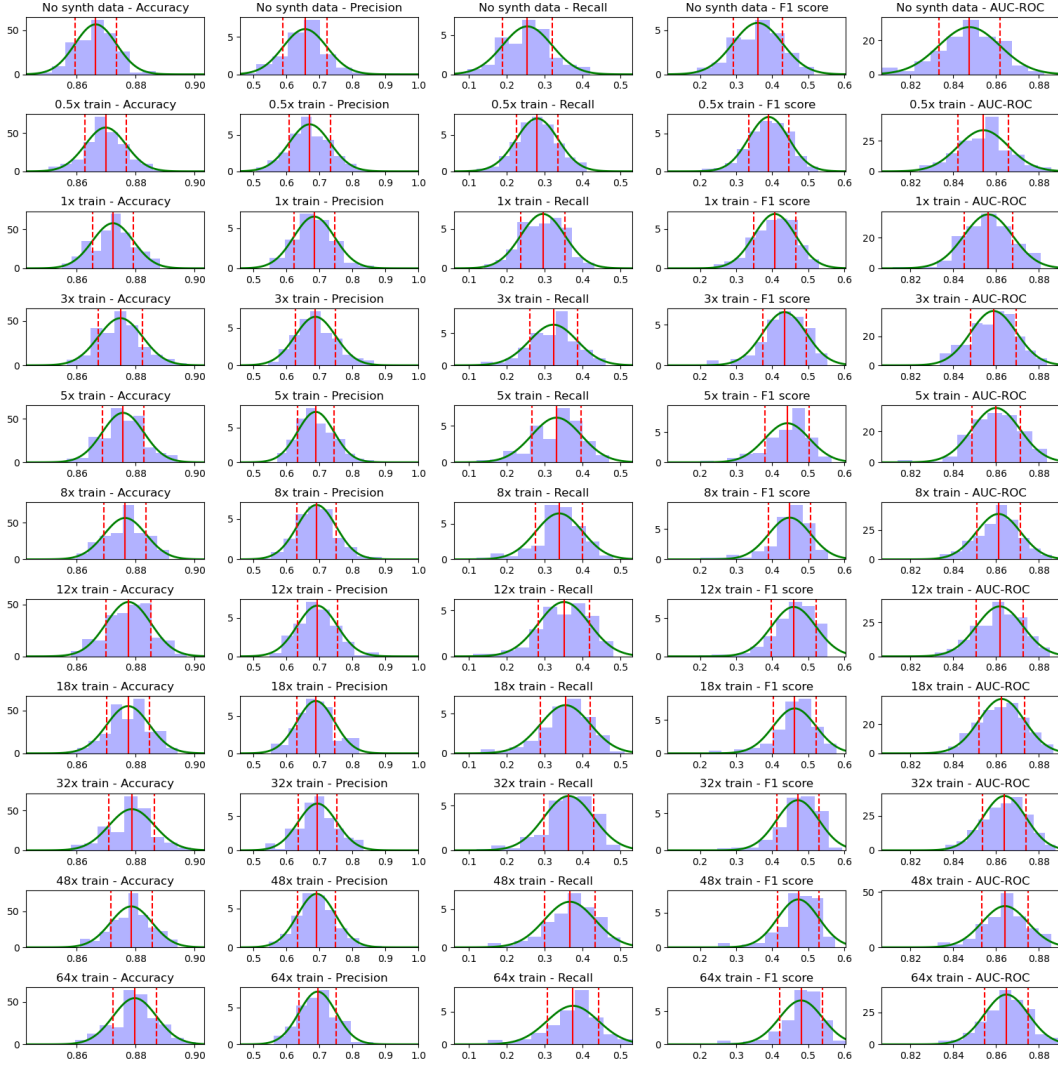
| Train set                        | Accuracy | Precision | Recall | F1     | AUC-ROC |
|----------------------------------|----------|-----------|--------|--------|---------|
| 1544 real obs.                   | 87.60%   | 67.30%    | 35%    | 46.00% | 87.30%  |
| 1544 real obs. + 99k synth. obs. | 87.80%   | 67.10%    | 39.50% | 49.70% | 86.40%  |
| Difference                       | 0.20%    | -0.20%    | 4.50%  | 3.70%  | -0.90%  |

## 5.4. Results for the External Approach, Case I

Recall from methodology section 4.3.2 that for the external approach we will consider two sizes for the small bootstrap train set. The first case we consider, “case I”, assumes a small train set with a size equal to 15% of the total dataset. The results for adding synthetic data in case I are given in figure 5.6 below. Looking at the line plots, we see that the median of the bootstrapped performance metrics clearly increases when synthetic data is added. We can also see that the performance increase diminishes when adding more and more synthetic data, similar to what we saw in literature. Again, there is quite some overlap between the errorbars, but improvement in performance is distinct, especially when considering the bootstrap distributions in figure 5.7 on the next page. In table 5.3 we show the absolute performance difference between adding 0 and 99k synthetic datapoints. When adding 99k synthetic datapoints we get performance metrics that are on par with the performance metrics when using only the full real training dataset to train the RF as we did in the baseline approach, see table 5.1. We can thus argue that we are able to replace real data by synthetic data without compromising on RF performance, which is a testament to the quality of the CTGAN generator. Do note that the distributions of the baseline figures are considerably more narrow, so we can not conclude this with certainty.



**Figure 5.6:** Results for the first case of the external approach. The small real training set has a size equal to 15% of total dataset, i.e. there are 331 real observations to train the RF when the synthetic data count equals 0 in the leftmost estimate in the figures. The amount of synthetic data added to the real set is a multiple of the original training set size, the multiples are 0.5x, 1x, 3x, 5x, 8x, 12x, 18x, 32x, 48x and 64x. For every metric, the dots represent the median of the 200 bootstrap iterations, and they are given as a function of the amount of synthetic data added. The errorbars are provided in the form of the 2.5% lower and 97.5% upper quantiles.



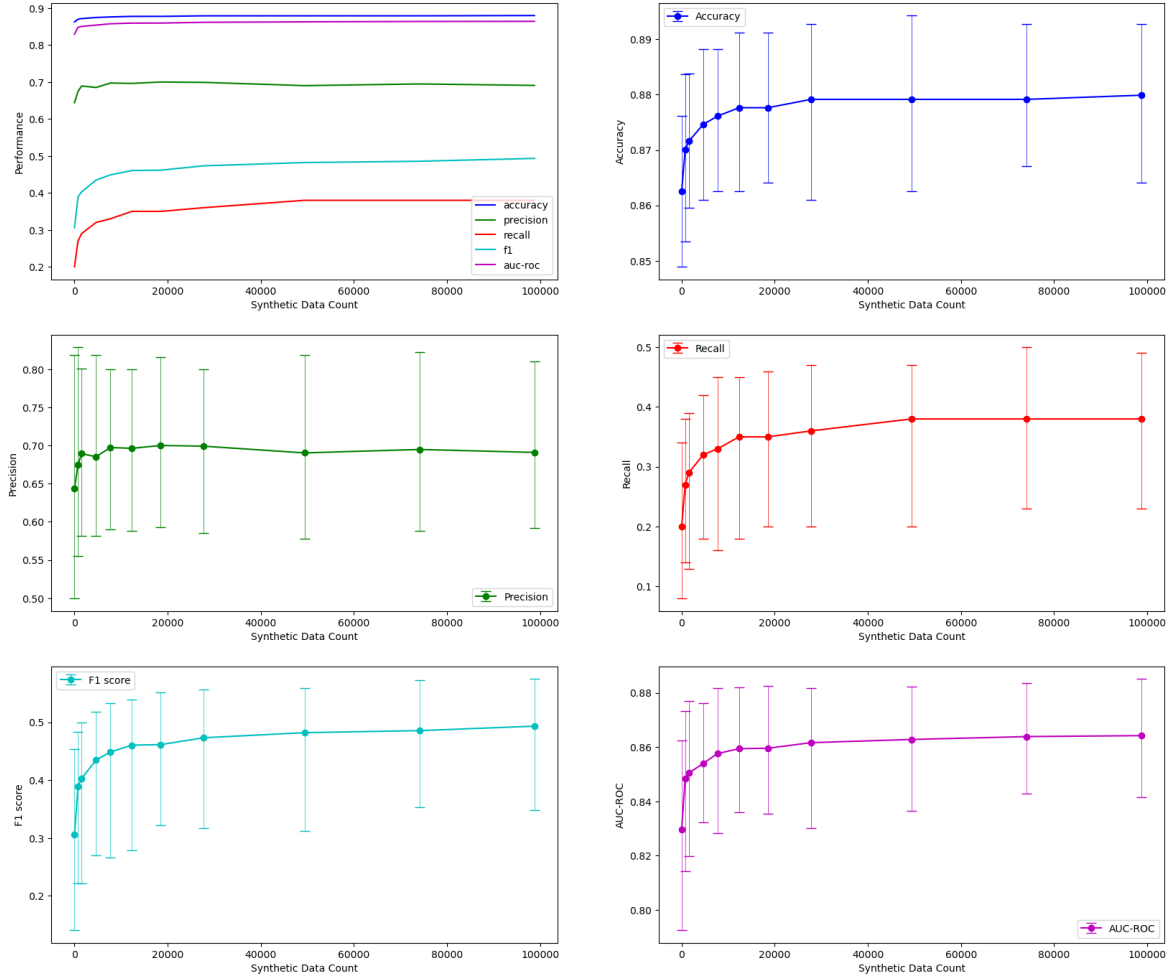
**Figure 5.7:** Histograms of the 200-iteration bootstraps for estimating the RF performance metrics for case I of the external approach. The x-axes are fixed for every metric, such that we can better observe distributional shifts. A normal density function is fitted in green and the mean and standard deviations are given by the solid and dotted red lines, respectively. Performance peaks when the synthetic set is equal to 64x the size of the original train set (i.e. 99k synthetic observations). When comparing the top and bottom figures we can clearly see that the distributions shift to the right for all performance metrics.

**Table 5.3:** Difference in the median performance between using only the small real train set with 331 observations, versus adding 99k synthetic datapoints to the small real train set.

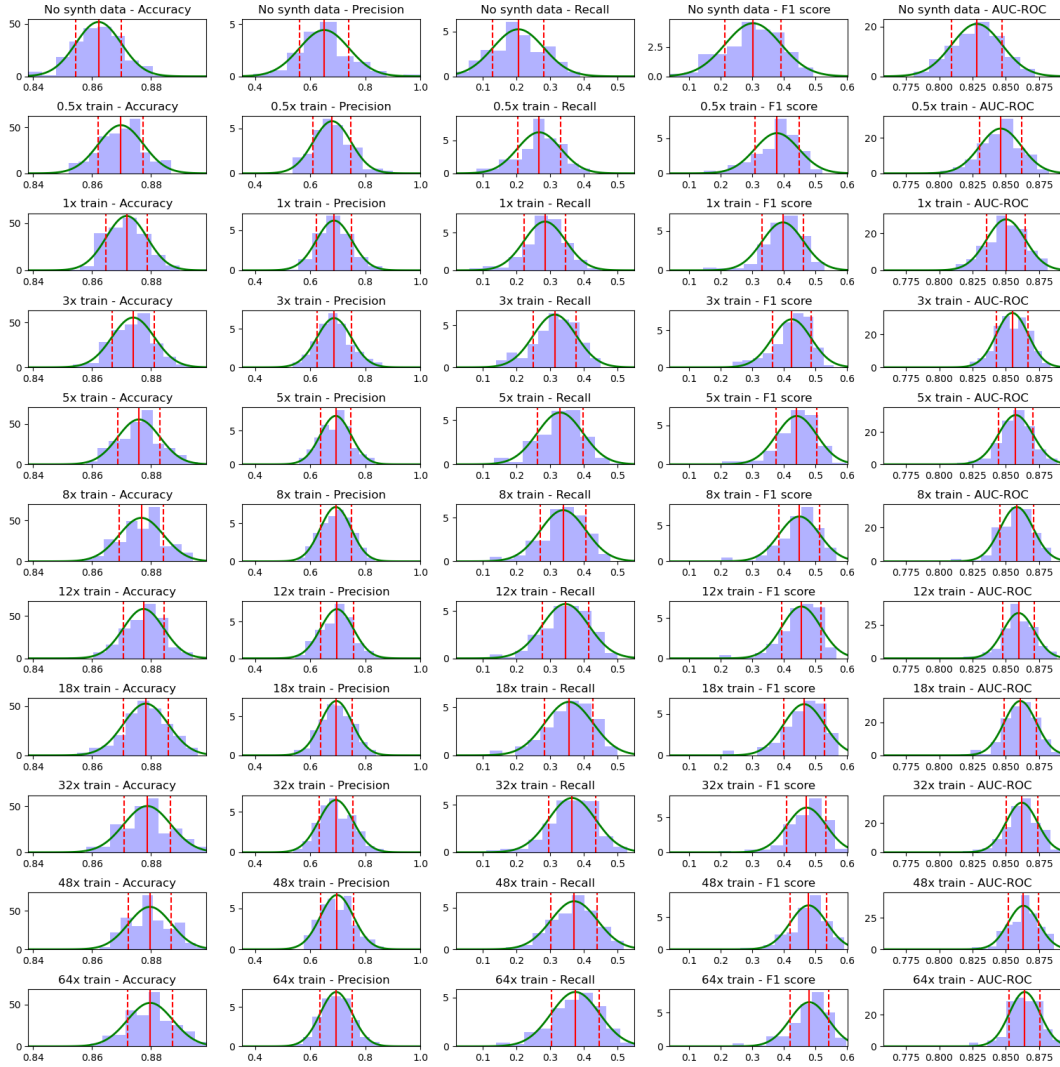
| Train set                       | Accuracy | Precision | Recall | F1    | AUC-ROC |
|---------------------------------|----------|-----------|--------|-------|---------|
| 331 real obs.                   | 86.7%    | 65.9%     | 26.0%  | 36.6% | 84.9%   |
| 331 real obs. + 99k synth. obs. | 88.0%    | 69.5%     | 38.0%  | 49.4% | 86.6%   |
| Difference                      | 1.3%     | 3.6%      | 12.0%  | 12.8% | 1.7%    |

## 5.5. Results for the External Approach, Case II

Next we consider external case II, where we assume the small training set has a size equal to 7.5% of the total dataset. Again we add synthetic data and check how this impacts the RF metrics, see figure 5.8 below. The characteristics of improvement are similar to those observed in case I, but for case II they are more pronounced. This is as expected, because we start out with a smaller real train set of size 7.5%, which performs worse than the set of size 15%. An interesting observation in table 5.4 on the next page is that, although we use less real data in this case, we are still able to boost performance to a level that is on par with case I and the baseline case. The histograms for case II are also given on the next page.



**Figure 5.8:** Results for the second case of the external approach. The small real training set has a size equal to 7.5% of total dataset, i.e. there are 165 real observations to train the RF when the synthetic data count equals 0 in the leftmost estimate in the figures. The amount of synthetic data added to the real set is a multiple of the original training set size, the multiples are 0.5x, 1x, 3x, 5x, 8x, 12x, 18x, 32x, 48x and 64x. For every metric, the dots represent the median of the 200 bootstrap iterations, and they are given as a function of the amount of synthetic data added. The errorbars are provided in the form of the 2.5% lower and 97.5% upper quantiles.



**Figure 5.9:** Histograms of the 200-iteration bootstraps for estimating the RF performance metrics for case II of the external approach. The x-axes are fixed for every metric, such that we can better observe distributional shifts. A normal density function is fitted in green and the mean and standard deviations are given by the solid and dotted red lines, respectively. Performance peaks when the synthetic set is equal to 64x the size of the original train set (i.e. 99k synthetic observations). When comparing the top and bottom figures we can clearly see that the distributions shift to the right for all performance metrics.

**Table 5.4:** Difference in the median performance between using only the small real train set with 165 observations, versus adding 99k synthetic datapoints to the small real train set.

| Train set                       | Accuracy | Precision | Recall | F1    | AUC-ROC |
|---------------------------------|----------|-----------|--------|-------|---------|
| 165 real obs.                   | 86.3%    | 64.4%     | 20.0%  | 30.6% | 83.0%   |
| 165 real obs. + 99k synth. obs. | 88.0%    | 69.1%     | 38.0%  | 49.3% | 86.4%   |
| Difference                      | 1.7%     | 4.7%      | 18.0%  | 18.7% | 3.5%    |

Going back to the business problem at hand, we summarize the revenue, cost and profit numbers in table 5.5 below. The algorithms trained on the different datasets we considered all turn a positive profit. This is expected, as they all have a precision of well above 27.27%, which was the baseline requirement for generating profit. In fact, precision is fairly high for all train sets that we considered. As we pointed out earlier, the marginal profit contribution of precision diminishes as precision get higher. We would therefore benefit more from an increase in recall. Interestingly, recall is the metric that improves the most when increasing the size of the real train data set or when adding synthetic data. We can further see from the table that using the full training set, or using a small training set complemented with synthetic data, leads to the highest profit. It is important to point out that the difference in revenue between the 3rd, 4th, 5th and 6th rows are **not** significant, as their errorbars and bootstrap distributions overlap significantly. Instead, we assume that these datasets turn a comparable profit, and their profit is significantly better than the profit of using only the small datasets in row 1 and 2. For external case I, adding the synthetic data increases profit by 51%, for external case II it is increased by 99%. Lastly, note that the profit margins are comparable for all train sets considered.

**Table 5.5:** Revenue, costs, profit, and profit margin for the different train sets considered in this thesis. Note: the numbers assume a marketing campaign in which 2205 customers participated. For every train set listed we looked at the associated median recall and precision values of the algorithms that were trained on it, and the results are obtained by plugging in these recall and precision values in equation 4.9 and 4.10

| Train set                                      | Revenue    | Costs    | Profit   | Margin |
|------------------------------------------------|------------|----------|----------|--------|
| 165 real obs. (7.5% of total set)              | \$732.50   | \$310.20 | \$422.30 | 57.7%  |
| 331 real obs. (15% of total set)               | \$952.25   | \$394.09 | \$558.16 | 58.6%  |
| 1544 real obs. (70% of total set)              | \$1,281.88 | \$519.47 | \$762.41 | 59.5%  |
| 165 real obs. + 99k synth. obs. (Ext. case II) | \$1,391.75 | \$549.38 | \$842.38 | 60.5%  |
| 331 real obs. + 99k synth. obs. (Ext. case I)  | \$1,391.75 | \$546.14 | \$845.61 | 60.8%  |
| 1544 real obs. + 99k synth. obs. (Int. case)   | \$1,446.69 | \$588.01 | \$858.68 | 59.4%  |

The results that we discussed in this chapter are largely in line with our expectations, most notably: (i) synthetic data does not meaningfully impact RF performance in the internal approach, and (ii) synthetic data does improve RF performance in the external approach. Given (i), we can also conclude that our hypothesis that synthetic data can internally boost a tabular dataset by adding variability and accounting for underrepresented areas in the feature space is not proven. Note that this result is different from e.g. the imagery data literature, where improvements are usually observed when synthetically extending datasets. It may still be possible that for datasets with an alternative structure, e.g. heavily imbalanced datasets, the internal approach does boost performance. Result (ii) is a very promising result for synthetic data sharing situations. The fact that performance was not only improved, but that it improved to the same level as when one would use the full real train set makes the result extra interesting. This result also is a testament to the quality of the synthetic data that the CTGAN generated, as we can apparently replace the real data with synthetic data without compromising on the RF's performance.

## 6. Conclusion

This research set out to investigate whether synthetically extending a tabular marketing dataset using the CTGAN generator can boost the performance of a random forest (RF) algorithm when the algorithm is trained on the extended set. To determine this, we implemented the CTGAN in Python and developed two different methodologies for training RF algorithms. These methodologies combined with a bootstrap approach allowed us to robustly assess how synthetically extending our tabular dataset impacts RF performance.

The first method that we considered was the internal approach. Here, the real training dataset is synthetically extended by a CTGAN that is trained on the very same training set. The hypothesis is that synthetic data may boost the RF performance by acting as a regularizer by increasing the variety of the feature space and by filling areas in the feature space that are underrepresented – we do **not** assume that synthetic data can add information that was not already present in the real dataset itself. Although this approach does show performance improvements when applied to e.g. imagery data, in our research we did not find a statistically significant indication that the internal approach improves (or decreases) the performance of the RF algorithm.

The second, more compelling method that we considered was the external approach. Here, we consider a training set that is resampled to create a small training set for RF training and a large training set for CTGAN training. The CTGAN then generates synthetic data that is added to the smaller, real training set. Consequently, the largest part of the original training set is fed to the RF algorithm in synthetic form. This approach is of interest for two reasons: (i) we can check how replacing real data by synthetic data impacts RF performance, and (ii) it replicates a hypothetical situation where a company with limited data access can buy additional data in synthetic form. In the external approach it was found that adding synthetic data clearly improves the RF performance – the smaller the real training dataset considered, the greater the improvement when synthetic data is added. The smallest real train set we considered is equal to 7.5% of the total dataset (165 obs.), and when adding 99k synthetic datapoints to this small train set the absolute increase in accuracy, precision, recall, F1 and AUC-ROC is 1.7%, 4.7%, 18.0%, 18.7% and 3.5%, respectively. We showed how precision and recall determine our algorithm’s profit generating ability. The reported increase in these metrics grows the profit from \$422.30 to \$842.38, which is almost a 100% increase. Another interesting result is that adding the synthetic data to the small dataset boosts the RF’s performance to a level that is on par with the performance of using the full, real training set. This result indicates that the CTGAN generator creates synthetic data of considerable quality, as we are effectively able to replace real data by synthetic data without compromising on the RF’s performance.

There are a few angles to consider for further research. First of all, we can improve the estimates of the RF performance by simulating more bootstrap samples. This would allow us to create more reliable distributions for the performance metrics, and in turn would make it possible to do statistical inference more thoroughly. Moreover, we only considered a single train-test split in our research. Different splits may lead to different results, so it would be worthwhile to investigate a few other splits in e.g. a cross-validation approach. In line with this reasoning, it would also be interesting to look at a totally different dataset. For example, a heavily imbalanced dataset may respond differently to the addition of synthetic data. Lastly, we could do more research into the synthetic data generator: we can perform statistical test to assess the quality of the synthetic data, we could investigate possible modifications to improve the CTGAN and we could look at alternative generative models like the Tabular Variational Auto-Encoder or a diffusion model, to see if they perform better.

# Bibliography

- [1] Arthur, C. (2013). Tech giants may be huge, but nothing matches big data. The Guardian. the-guardian.com
- [2] Weldon, J., Ward, T., & Brophy, E. (2021). Generation of synthetic electronic health records using a federated GAN. arXiv preprint arXiv:2109.02543. <https://doi.org/10.48550/arXiv.2109.02543>
- [3] Frid-Adar, M., Diamant, I., Klang, E., Amitai, M., Goldberger, J., & Greenspan, H. (2018). GAN-based synthetic medical image augmentation for increased CNN performance in liver lesion classification. *Neurocomputing*, 321, 321-331. <https://doi.org/10.1016/j.neucom.2018.09.013>
- [4] Borkman, S., Crespi, A., Dhakad, S., Ganguly, S., Hogins, J., Jhang, Y. C., Kamalzadeh, M., Li, B., Leal, S., Romero, C., Smith, W., Thaman, A., Warren, S., & Yadav, N. (2021). Unity perception: Generate synthetic data for computer vision. arXiv preprint arXiv:2107.04259. <https://doi.org/10.48550/arXiv.2107.04259>
- [5] Gartner Inc. (2021). By 2024, 60% of the data used for the development of AI and analytics projects will be synthetically generated. Retrieved from gartner.com
- [6] Funding data. Retrieved from crunchbase.com
- [7] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S. & Bengio, Y. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27. <https://doi.org/10.48550/arXiv.1406.2661>
- [8] Qian, Z., Cebere, B. C., & van der Schaar, M. (2023). Synthcity: facilitating innovative use cases of synthetic data in different data modalities. arXiv preprint arXiv:2301.07573. <https://doi.org/10.48550/arXiv.2301.07573>
- [9] Gunasekar, S., Zhang, Y., Aneja, J., Mendes, C. C. T., Del Giorno, A., Gopi, S., Javaheripi, M., Kauffman, P., De Rosa, G., Saarikivi, O., Salim, A., Shah, S., Singh Behl, H., Wang, X., Bubeck, S., Eldan, R., Tauman Kalai, A., Tat Lee, & Li, Y. (2023). Textbooks Are All You Need. arXiv preprint arXiv:2306.11644. <https://doi.org/10.48550/arXiv.2306.11644>
- [10] Manousakas, D., & Aydıre, S. (2023). On the Usefulness of Synthetic Tabular Data Generation. arXiv preprint arXiv:2306.15636. <https://doi.org/10.48550/arXiv.2306.15636>
- [11] Yoon, J., Jordon, J., & Schaar, M. (2018, July). RadialGAN: Leveraging multiple datasets to improve target-specific predictive models using Generative Adversarial Networks. In *International Conference on Machine Learning* (pp. 5699-5707). PMLR. <https://doi.org/10.48550/arXiv.1802.06403>
- [12] Dina, A. S., Siddique, A. B., & Manivannan, D. (2022). Effect of balancing data using synthetic data on the performance of machine learning classifiers for intrusion detection in computer networks. *IEEE Access*, 10, 96731-96747. <https://doi.org/10.48550/arXiv.2204.00144>
- [13] ML model performance as function of training data. Retrieved from mostly.ai.
- [14] Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling tabular data using conditional gan. *Advances in Neural Information Processing Systems*, 32. <https://doi.org/10.48550/arXiv.1907.00503>

- [15] van Breugel, B., Qian, Z., & van der Schaar, M. (2023). Synthetic data, real errors: how (not) to publish and use synthetic data. arXiv preprint arXiv:2305.09235. <https://doi.org/10.48550/arXiv.2305.09235>
- [16] Marketing Campaign [Data set]. Retrieved from Kaggle.com. Kaggle.com
- [17] Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. C. (2017). Improved training of wasserstein gans. Advances in Neural Information Processing Systems, 30. <https://doi.org/10.48550/arXiv.1704.00028>